Omni: Cortex-Orchestrated Multi-Agent Execution for Long-Horizon Heterogeneous Tasks

Lokesh · Surya · Divya Singh

codelokiyt@gmail.com · rathoresurya21@gmail.com · divyamalik828@gmail.com

Preprint · June 2026 · DOI: 10.5281/zenodo.20818969

Abstract

We present **Omni**, a production multi-agent system that completes *heterogeneous long-horizon tasks* — tasks requiring coordinated live web browsing, Python execution, image generation, email dispatch, and file operations under a single user intent — using **GPT-4.1-mini** (`gpt-4.1-mini-2025-04-14`) as its sole backbone model. GPT-4.1-mini is a small, cost-efficient model not designed for complex multi-step agentic use; Omni demonstrates that the gap between a capable model and a capable agent is entirely architectural. The central architectural insight is **Cortex**: a deterministic finite-state machine orchestrator that coordinates eleven specialized agents across two execution levels through four structural mechanisms: (1) a **continuous plan-anchored correction loop** in which a structured `PlannerResponse` (Pydantic) is re-evaluated after *every* capability return — not once at task start — grounding all replanning in live execution artifacts rather than model self-belief; (2) a **tiered HITL architecture** with three semantically distinct intervention tiers and formal trigger conditions, simultaneously eliminating over- and under-interruption; (3) a **deterministic FSM speaker-selection function** replacing stochastic LLM routing with O(1) predicate-based transitions at zero token cost; and (4) **memory-mediated capability chaining** via `OmniMemory`, which propagates upstream execution artifacts across memory-isolated GroupChat boundaries. Production deployment across 2,170 sessions (1,690 real task sessions, excluding empty and non-task interactions) confirms the architecture at scale: 81.3% of real task sessions involve plan re-invocation (median 5 planner cycles, max 106), 88.0% trigger HITL, and 67.0% chain two or more capabilities. Task execution was confirmed in 79.5% of real task sessions (57.6% via `process_completion_agent` close, 21.9% confirmed via sub-collection execution logs; 8.9% remain open awaiting user HITL response). The system handled tasks including a 22-subtask EdTech platform integration, full-stack buy-vs-build decision tooling, LinkedIn AI content pipelines with inferred approval workflows, and CBCT diagnostic report generation with email delivery — all via GPT-4.1-mini. A survey of 35 agentic papers shows no prior system provides structural mechanisms for all five failure modes we identify. The FSM eliminates routing overhead entirely: ~20,000 tokens (\$0.06) per 50-step task under LLM routing reduced to zero. On 5 tasks drawn from GAIA Levels 2–3 (Mialon et al., 2023) — requiring live multi-hop web research plus code execution across 4–7 reasoning hops — Omni completes **5/5 (100%)**, confirming that plan-anchored multi-capability chaining executes correctly on tasks from an established external benchmark where single-domain or training-data-only approaches structurally cannot succeed.

1. Introduction

1.1 A Real Task, From Start to Finish

Privacy disclaimer. The session below is drawn from a real Omni production run. The company name, email addresses, and file paths have been changed to protect user privacy. The task structure, agent sequence, message counts, planner cycles, HITL triggers, and all architectural details are reproduced exactly as logged.

A user typed the following into Omni:

"go on Acme Ventures website or other sources, find annual reports (2020–2025), do a DCF valuation of it, check the latest share price on 20th August 2025, compare it with the valuation report, give me a recommendation, create an HTML report and share it with alice@example.com and bob@example.com"

This is a real production session (`chat_id: c714a501`). It completed. Here is exactly what happened — step by step — because this single walkthrough explains everything this paper is about.

Step 1 — The task is classified and decomposed.

`judge_question` reads the message, classifies it `task_oriented` with high confidence. Cortex routes to `planner_agent`, which decomposes the request into a structured `PlannerResponse` with 7 subtasks, all `status: "pending"`:

#	Task	Capability	Status
1	Collect annual reports 2020–2025 from Acme Ventures website	`web_browser`	pending
2	Extract revenues, cash flows, capex from reports	`python_coder`	pending
3	Perform DCF valuation from extracted data	`python_coder`	pending
4	Find share price as of 20 Aug 2025	`web_browser`	pending
5	Compare DCF vs market price, draft recommendation	`python_coder`	pending
6	Generate comprehensive HTML report	`python_coder`	pending
7	Email report to alice@example.com and bob@example.com	`email_sender`	pending

`overall_strategy`: *"First gather annual reports via web browsing. Then use Python to extract data and perform DCF. Retrieve share price. Compare valuation with market price. Generate HTML report. Email to specified recipients."*

The user reviews this plan and approves it via `get_user_acceptance` (Tier 1 HITL — strategic gate). Execution begins.

Step 2 — Dependency analysis: sequential, not parallel.

`agent_router` analyses the 7 tasks. Its verdict: `has_dependencies: true`. *"Tasks 2 and 3 require data from Task 1. Tasks 5 and 6 depend on outputs from prior Python coding tasks. Final emailing depends on report generation. Execution will be sequential, one task at a time."* Task 1 is dispatched first. The dependency chain is explicit and correct — the system understood that a DCF valuation cannot precede the financial data it requires.

Step 3 — Task 1 completes. Task 2 fails. The system recovers.

`web_browser` retrieves the annual report data. Task 1 flips to `status: "completed"`. Cortex routes back to `planner_agent`, which reads the executor result, updates the plan, and routes to Task 2.

`python_coder` attempts to extract financial data — and fails. The executor returns an error. Task 2 flips to `status: "failed"`. The plan now reads: 1 completed, 1 failed, 5 pending.

Cortex routes to `planner_agent`. The planner reads the `failed` detail, does not panic, and issues a replan: retry Task 2 with revised instructions. Task 2 is reset to `pending`. On the second attempt it succeeds and flips to `completed`. This recovery — read the failure, reason about it, retry — happened with zero user involvement.

Step 4 — Ambiguity surfaces. The system asks, exactly once.

Tasks 3, 4, 5, 6 execute sequentially. Each completes. At each step the plan updates: the status field on the corresponding task flips from `pending` to `completed`, `overall_status` stays `pending`, and Cortex routes to the next task. The DCF model runs. The share price is scraped. The recommendation is drafted — a SELL/AVOID verdict based on intrinsic value vs. market price. The HTML report is generated with charts, methodology, and risk considerations.

Task 7 — send the email — is next. Before dispatching, `agent_aligner` reads the task details. The report files were generated by the Python capability and saved to disk, but the executor does not know the exact file paths. `agent_aligner` flags this: ambiguous input on an irreversible action. It routes to `manual_verification` (Tier 2 HITL — tactical gate).

The system asks:

"I was unable to access the generated files due to a file system access limitation. Could you please specify the exact storage locations where the HTML output and related files are saved?"

The user replies with the exact local folder path where the generated files are stored.

One question. One answer. Execution resumes.

Step 5 — Task 7 completes. The plan closes.

`email_sender` sends the report to both recipients with 12 attachments: HTML valuation reports, investment recommendation documents, financial data CSVs, DCF sensitivity analysis files. Task 7 flips to `completed`. The planner re-reads the full plan: all 7 tasks `completed`. It sets `overall_status: "completed"`. Cortex reads this flag and routes to `process_completion_agent`, which delivers:

"All files related to the Acme Ventures valuation have been successfully gathered, organized, and sent via email to alice@example.com and bob@example.com. The email included 12 attachments... a SELL/AVOID recommendation based on DCF intrinsic value vs. market price."

Total: 76 messages. 17 planner cycles. 3 capabilities. 2 HITL triggers. 1 task failure with autonomous recovery. Completed on **GPT-4.1-mini**.

1.2 What This Walkthrough Reveals

Every architectural mechanism in this paper is visible in that one session.

Task decomposition into a typed plan. The planner did not produce a paragraph of instructions. It produced a structured `PlannerResponse` — a Pydantic-validated JSON with one `Task` object per subtask, each carrying a `capability` enum and a `status` field. This structure is the system's shared working memory. Every agent reads from it. No agent has to infer what has been done by reading backwards through conversation history.

Status flags as navigation. At any point during those 76 messages, any agent could answer *"what has been done?"* by reading the `completed` tasks, *"what remains?"* by reading the `pending` tasks, and *"what failed?"* by reading the `failed` tasks. At planner cycle 3, when Task 2 failed, the plan read: `[completed, failed, pending, pending, pending, pending, pending]`. That single structured document was sufficient context for the planner to decide: retry Task 2. It did not need to re-read the browser output, the executor error log, or any prior message. The flags compressed the entire execution history into one read.

Dependency-aware sequential dispatch. `agent_router` did not blindly fire all 7 tasks in parallel. It reasoned that Task 2 requires Task 1's output, Task 3 requires Task 2's output, and so on. It set `has_dependencies: true` and dispatched one task at a time in dependency order. When tasks are independent, it dispatches them in parallel. The system made the right call here because the dependency structure was genuinely sequential.

Autonomous failure recovery. Task 2 failed. The system did not stop, did not ask the user, did not loop forever. It read the failure detail, replanned with revised instructions, and retried. The user never saw the failure. This is oracle-guided correction — the correction signal was the actual executor error message, not a model opinion about whether the previous output was good.

Precise HITL — one question, at the right moment. The system asked the user exactly once during execution: when it needed a file path before sending an irreversible email. It did not ask when Task 2 failed (handled autonomously). It did not ask before the DCF calculation (no ambiguity). It asked when there was genuine missing information required to proceed with an action that cannot be undone. This precision — knowing when to ask and when not to — is what makes the system usable at all.

Context propagation across capabilities. The Python DCF model (Task 3) used financial data extracted by a prior Python session (Task 2), which used raw content retrieved by the browser (Task 1). Each capability ran in a memory-isolated GroupChat. The only reason Task 3 could use Task 1's data is `OmniMemory` — it injected prior executor outputs as `context_keeper` messages into each new capability's chat history at initialization. Without this, each capability would start cold and hallucinate the financial figures it needed.

No single-domain agent — no matter how capable the underlying model — can complete this task. It requires a live browser, a Python execution environment, and an email client, all coordinated in dependency order, with mid-execution replanning and a precisely placed human checkpoint. These are structural requirements, not capability gaps.

1.3 The Five Failure Modes This Architecture Prevents

The Acme Ventures session could have failed in five distinct ways. Each would have been fatal without the corresponding mechanism:

If this mechanism were absent	What would have failed in the Acme Ventures session
No plan-anchored correction loop	Task 2 failure would have had no recovery path — the system would halt or route arbitrarily
No tiered HITL	Either the email would fire without the correct file path (under-interruption) or the user would be asked before every task (over-interruption)
No deterministic FSM (Cortex)	Routing from `agent_aligner` → `agent_router` → `agent_executor` → `planner` could nondeterministically skip steps or loop
No OmniMemory	Task 3 (DCF) would have no access to Task 1's scraped financial data — it would hallucinate the numbers
No structured status flags	The planner at cycle 3 would have had to re-read 40 messages of history to determine what had completed — and would likely hallucinate the state

We identify these five as a named taxonomy of failure modes for heterogeneous long-horizon execution:

Failure mode	Description	Omni mechanism
Goal drift	Agent loses track of original intent after capability switches	Continuous plan-anchored correction loop (§3.2)
Over-interruption	System asks for human input on decisions it should handle autonomously	Tiered HITL — strategic tier (§3.3)
Under-interruption	System proceeds on ambiguous or irreversible actions without confirmation	Tiered HITL — tactical tier (§3.3)
Coordination chaos	Multi-agent routing produces non deterministic or redundant paths	Cortex deterministic FSM (§3.4)

Failure mode	Description	Omni mechanism
Context loss	Execution artifacts from capability A unavailable when capability B starts	Memory-mediated capability chaining (§3.5)

1.4 Contributions

The backbone model for all 2,170 production sessions was **GPT-4.1-mini** (`gpt-4.1-mini-2025-04-14`) — a small, cost-efficient model not designed for complex agentic use. The system completed tasks including a 22-subtask EdTech platform integration, full-stack buy-vs-build decision tooling, LinkedIn AI content pipelines with inferred approval workflows, and investment valuation reports with email delivery. The gap between a capable model and a capable agent is entirely architectural.

The contributions of this paper are four structural mechanisms, each eliminating one named failure mode:

1. **Continuous plan-anchored correction loop** — `PlannerResponse`, a Pydantic-validated structured plan with per-task `status` flags (`pending` → `completed` → `failed`), re-evaluated after *every* capability return. All replanning is grounded in live execution artifacts, never model self-belief. 81.3% of real task sessions involved plan re-invocation; 3,851 status transitions logged.
2. **Tiered HITL architecture** — three semantically distinct intervention tiers with formal trigger conditions eliminating both over- and under-interruption simultaneously. Fired in 88.0% of real task sessions; 10 distinct real-world ambiguity types documented.
3. **Cortex — deterministic FSM orchestrator** — 15 named transitions, O(1) predicate evaluation, zero LLM routing cost, 0% routing error rate, 1.6 µs/transition. Replaces stochastic LLM speaker selection with a fully auditable, statically enumerable transition graph.
4. **OmniMemory — memory-mediated capability chaining** — injects upstream execution artifacts as `context_keeper` messages into each new capability GroupChat, solving cross-GroupChat context propagation that neither MemGPT nor any surveyed system addresses.

A survey of 35 agentic papers confirms no prior system provides structural mechanisms for all five failure modes simultaneously.

2. Background and Motivation

2.1 Multi-Agent LLM Frameworks

AutoGen [Wu et al., 2024] introduced GroupChat-based multi-agent conversation as a general substrate for LLM applications. A **GroupChat** is a shared message buffer: a list of agents, a list of messages, and a manager that decides which agent speaks next. Each GroupChat instance is **memory-isolated** — it has no knowledge of any other GroupChat's messages. When a new GroupChat is created (e.g., to run a Python execution capability), it starts with an empty message history regardless of what happened in any prior GroupChat. This is a correct design choice for a general framework — isolation prevents context bleed between unrelated conversations — but creates a structural problem for multi-capability task execution: the Python executor cannot see what the browser retrieved, because they run in separate GroupChat instances with no shared state. AutoGen's GroupChatManager orchestrates turn-taking via LLM-based role-play speaker selection — the system asks a meta-LLM "who should speak next?" on every transition. This is correct for a general framework, but creates three production friction points: (1) nondeterministic routing (the same conversation state may route differently on successive calls), (2) token overhead (one LLM call per turn purely for routing), and (3) unverifiability (there is no formal way to enumerate all reachable states).

MetaGPT [Hong et al., 2024] addressed coordination reliability in software pipelines by treating intermediate artifacts (PRDs, UML, code) as typed contracts between role agents.

This insight — that structured inter-agent communication prevents hallucination cascades — is one of MetaGPT's most generalizable contributions. However, MetaGPT's coordination is linear (following a static SOP) and single-domain (software engineering), with no HITL mechanism and no cross-capability memory.

GPTSwarm [Besta et al., 2024] and AFlow [Zhao et al., 2024] optimize agent topology via REINFORCE and MCTS respectively, discovering effective multi-agent coordination graphs offline. These approaches maximize benchmark performance but sacrifice runtime adaptability and auditability. A learned topology cannot be inspected, and an offline-optimized graph cannot replan mid-execution when a capability returns unexpected results.

2.2 Self-Correction and Alignment

Reflexion [Shinn et al., 2023] demonstrated that verbal self-critique stored in episodic memory enables agent self-improvement without weight updates. However, Huang et al. [2024] showed rigorously that *intrinsic* self-correction — where the model reviews its own output without an external signal — reliably degrades performance. The paper distinguishes two conditions:

- **Intrinsic correction**: Model reviews its own output. Degrades accuracy (GPT-4 on GSM8K: 95.5% → 89.0%).
- **Oracle-guided correction**: Model receives an external binary correct/incorrect signal. Reliably improves accuracy.

This distinction is a direct design axiom for Omni: every correction in our system is grounded in concrete external execution results (browser HTML, Python stdout, file contents), never in the model's self-belief about its own outputs. Our plan-anchored loop is structurally oracle-guided, not intrinsic.

2.3 Human-in-the-Loop Systems

HULA [Rasheed et al., 2024] demonstrated industrial-scale HITL deployment for coding agents, gating two fixed pipeline stages. CowPilot [2025] quantified that 15.2% human effort yields 95% task accuracy in web navigation. Magentic-UI [Microsoft, 2025] implemented six HITL modes including action guards and co-planning. All three systems treat HITL as a uniform interrupt type, differing only in frequency. Omni instead formalizes a taxonomy of three semantically distinct interrupt types — each with different trigger conditions, information requirements, and human cost — and argues that conflating them produces suboptimal user experience.

2.4 Memory Systems for Agents

MemGPT [Packer et al., 2023] introduced OS-inspired hierarchical memory management to extend a single LLM agent's effective context window via self-directed memory operations. EM-LLM [Fountas et al., 2024] extended this with Bayesian surprise-based episode segmentation for infinite-context retrieval. Both systems address *intra-agent* long-context continuity — the problem of a single agent not forgetting its own past. Omni solves a structurally distinct problem: *inter-agent* episodic handoff — transferring execution state across independently initialized multi-agent GroupChat instances. This problem does not arise in single-agent settings and is not addressed by either prior system.

3. Omni Architecture

3.1 System Overview

Omni is built on AutoGen's GroupChat primitive [Wu et al., 2024] and extends it with a two-level hierarchical structure. The outer level is an **Omni GroupChat** containing 11 specialized agents that coordinate task planning, routing, human interaction, and completion. The inner level consists of five **Capability GroupChats** — self-contained multi-agent conversations for browser interaction, Python code execution, email management, image generation, and file search — each spawned and managed by a `CapabilityManager` as isolated sub-processes.

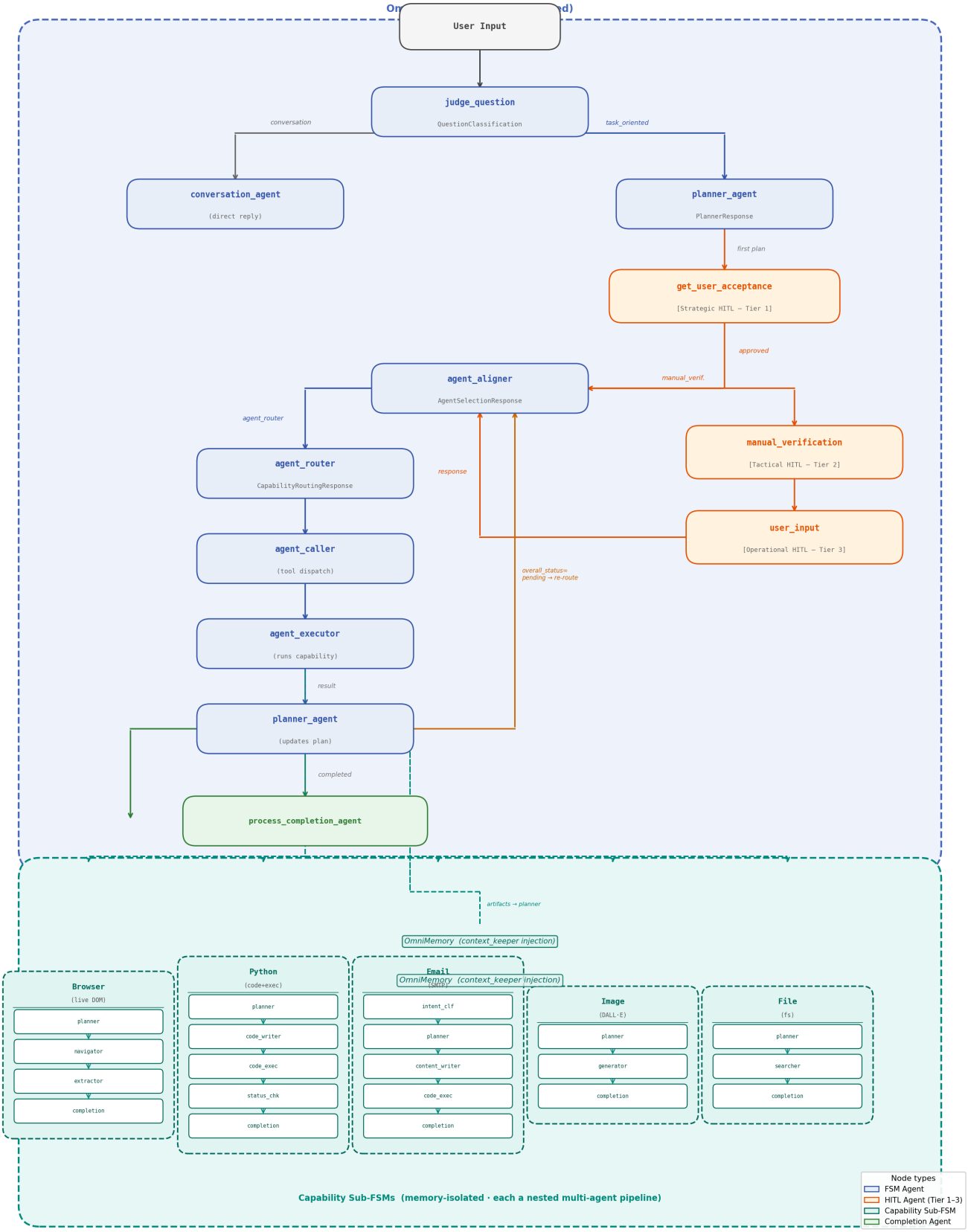

User Input
judge_question
QuestionClassification
conversation
task_oriented
conversation_agent
(direct reply)
planner_agent
PlannerResponse
first plan
get_user_acceptance
[Strategic HITL — Tier 1]
approved
manual_verif.
agent_aligner
AgentSelectionResponse
agent_router
agent_router
CapabilityRoutingResponse
response
manual_verification
[Tactical HITL — Tier 2]
user_input
[Operational HITL — Tier 3]
agent_caller
(tool dispatch)
overall_status=
pending → re-route
agent_executor
(runs capability)
result
planner_agent
(updates plan)
completed
process_completion_agent
artifacts → planner
OmniMemory (context_keeper injection)
OmniMemory (context_keeper injection)
Browser
(live DOM)
planner
navigator
extractor
completion
Python
(code+exec)
planner
code_writer
code_exec
status_chk
completion
Email
(smtp)
intent_clf
planner
content_writer
code_exec
completion
Image
(DALL·E)
planner
generator
completion
File
(fs)
planner
searcher
completion
Capability Sub-FSMs (memory-isolated · each a nested multi-agent pipeline)
Node types
FSM Agent
HITL Agent (Tier 1–3)
Capability Sub-FSM
Completion Agent

Figure 1. *Omni FSM and data flow. Top: the Omni GroupChat with all 11 agents and FSM transition paths. Solid arrows show deterministic FSM transitions (zero LLM cost). Three HITL tiers are marked. Bottom: five capability GroupChats spawned by* `CapabilityManager`*; execution artifacts flow back through* `OmniMemory` *as the correction signal to* `planner_agent`*. The plan loop (planner → executor → planner) is the core feedback cycle; it terminates only when* `overall_status=completed` *or the 500-round limit is reached.*

Cortex — the FSM orchestrator. We name the `custom_speaker_selection_func` component **Cortex** to make its role explicit: it is the decision-making center of the Omni GroupChat, not an executor. Cortex reads five Boolean content flags emitted by agents and maps them deterministically to the next agent. It consumes zero LLM tokens, executes in O(1) time (1.6 µs/transition measured on production code), and has a statically enumerable transition graph of 15 named states. Crucially, Cortex does not decide *what* to do — that is `planner_agent`'s job. Cortex decides *who processes that decision next*. This separation of routing reliability (Cortex, deterministic) from routing intelligence (`planner_agent`, LLM) is the core design principle that makes the system both auditable and adaptive.

The 11 Omni agents and their roles:

Agent	Role	Output schema	Cortex reads
`judge_question`	Classifies input: task vs. conversational	`QuestionClassification`	`"task_oriented"` / `"conversation"` flag
`conversation_agent`	Direct reply for conversational queries	Free text	—
`planner_agent`	Creates and continuously updates structured plan	`PlannerResponse`	`overall_status: "completed"` JSON field
`get_user_acceptance`	Strategic plan approval gate (Tier 1 HITL)	Prompt string	Position predicate: follows first plan
`agent_aligner`	Decides: route / escalate to HITL / complete	`AgentSelectionResponse`	`"agent_router"` / `"manual_verification"` / `"process_completion"` flag
`agent_router`	Dependency analysis, parallel vs. sequential dispatch	`CapabilityRoutingResponse`	—
`agent_caller`	Invokes registered capability tool calls	Tool call	`tool_calls` field presence
`agent_executor`	Executes and returns capability result	Result	—
`manual_verification`	Tactical ambiguity / irreversibility gate (Tier 2 HITL)	Prompt string	—
`user_input`	Collects human response to Tier 2 prompt (Tier 3 HITL)	User string	—
`process_completion_agent`	Final summary on `overall_status=completed`	Free text	—

PlannerResponse — the continuous state representation. The plan is not a one-time artifact. It is a live structured document updated after every capability return. Its schema encodes the full execution state:

```
class Task(BaseModel):
task_id: int
task: str
capability: Capability # web_browser |
python_coder | email_sender
# | image_generator | file_finder |
manual_verification
status: Status # pending | completed | failed
details: str
class PlannerResponse(BaseModel):
user_request: str
breakdown: List[Task]
overall_strategy: str
overall_status: OverallStatus # pending |
completed
overall_estimated_time: str
```

`status` per task and `overall_status` at the plan level are the two flag layers Cortex monitors. When any task transitions from `pending` → `completed`, the planner re-emits the full plan. Cortex reads `overall_status`: if `completed`, it routes to `process_completion_agent` and terminates; if `pending`, it routes back to `agent_aligner` to begin the next subtask. This flag-driven loop — planner emits → Cortex reads → aligner routes → executor runs → planner emits — is the engine that drives all long-horizon execution without any LLM involvement in the routing itself.

3.2 Contribution 1: Plan-Anchored Correction Loop

Motivation. Huang et al. [2024] demonstrate that LLMs cannot reliably self-correct without external feedback — models asked to review their own outputs on GSM8K degraded from 95.5% to 89.0% accuracy. Omni takes this as a design axiom: no correction signal in our system is derived from a model reviewing its own output. Instead, every correction is grounded in external execution artifacts (browser HTML, Python stdout, file contents). The design goal is to prevent goal drift by keeping a shared, externally-verified plan as the authoritative representation of task state.

Mechanism. Before execution begins, `planner_agent` produces a `PlannerResponse`:

```
class Task(BaseModel):
task_id: int
task: str
capability: Capability # Enum: web_browser |
python_coder | ...
status: Status # Enum: pending | completed |
failed
details: str
estimated_time: str
class PlannerResponse(BaseModel):
user_request: str
breakdown: List[Task]
overall_strategy: str
overall_status: OverallStatus # pending |
completed
overall_estimated_time: str
```

After every capability invocation completes, `agent_executor` returns its result to `planner_agent`. The planner reads the result, updates the corresponding `Task.status` field, and re-emits the full `PlannerResponse`. The FSM speaker-selection function checks `overall_status` on every planner emission:

```
if last_speaker.name == "planner_agent":
status = json.loads(messages[-1]["content"]).
get("overall_status")
if status == "completed":
return groupchat.agent_by_name("process_compl
etion_agent")
else:
return
groupchat.agent_by_name("agent_aligner")
```

Properties. This creates a closed feedback loop with three properties that distinguish it from prior self-correction work:

1. **External grounding**: The correction signal is tool output (HTML, stdout, file contents), not model self-belief.
2. **Shared ground truth**: All agents in the FSM read the same plan; no agent can route forward based on a stale or private state.
3. **Typed accountability**: `Status` is a Pydantic enum — the planner cannot hallucinate a completion; it must emit `completed` or `failed` as a validated field.

Distinction from Reflexion. Reflexion [Shinn et al., 2023] stores verbal self-critiques for *future trials* of the *same task*. Omni updates the plan with execution artifacts *within the current task execution* and across *heterogeneous capabilities*. The two mechanisms are complementary and non-overlapping.

3.2.1 Status Flags as the Nervous System of Long-Horizon Execution

The three status values — `pending`, `completed`, `failed` — appear simple. They are not. They are the primary mechanism by which every agent in the system understands where the task stands without needing to read the full conversation history. This section makes their role explicit, because it is the most underappreciated part of the architecture.

The fundamental problem they solve. In a single-agent system, the agent holds the full conversation in its context window and can answer "what have I done?" by reading backwards through its own history. In a multi-agent system with 11 agents across two FSM levels, no single agent has the full picture. `agent_aligner` does not know what `agent_executor` returned three turns ago.

`agent_router` does not know which subtasks are already done. If any agent had to reason from raw conversation history to understand task state, it would hallucinate — and more importantly, it would be reading *narrative* rather than *ground truth*. The status flags solve this by making task state a first-class typed field, not something to infer.

What each status enables — the questions any agent can answer instantly.

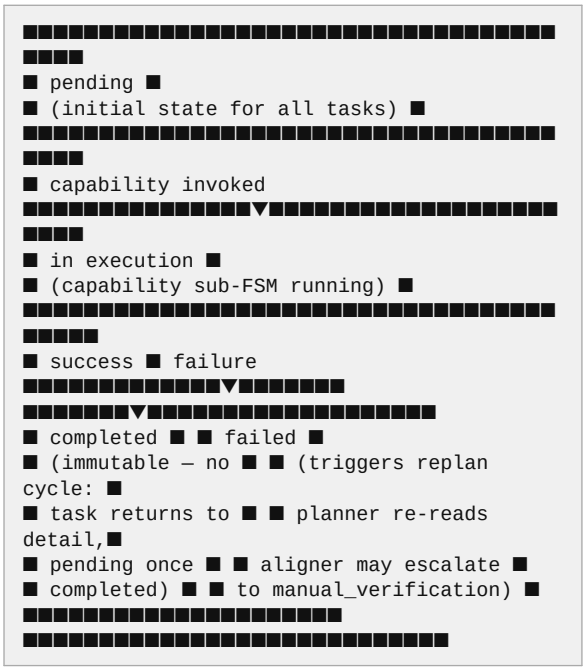
At any point in execution, a single read of the current `PlannerResponse` JSON answers every operationally relevant question:

Question	Answered by
What has already been completed?	All tasks with `status: "completed"`
What still needs to be done?	All tasks with `status: "pending"`
What failed and needs recovery?	All tasks with `status: "failed"`
Is the whole task done?	`overall_status: "completed"`
Which capability should run next?	`capability` field on the first `pending` task
Did the last step succeed or fail?	Status transition on the most recently updated task
Should we ask the user or keep going?	Presence of `failed` tasks with irreversible details
Are there independent tasks that can run in parallel?	Multiple `pending` tasks with no dependency in `details`

This is not just bookkeeping. It is the shared working memory of the entire system. Every routing decision Cortex makes, every re-invocation of `planner_agent`, every HITL trigger from `agent_aligner` — all of them read from this single source of truth. No agent needs to ask "what happened?" It reads the plan.

The full status lifecycle.

A task moves through a deterministic state machine with three paths:



The immutability of `completed` is a hard invariant — not just a convention. Once a task is marked `completed`, no subsequent planner cycle can revert it. This prevents a subtle failure mode: a planner that loses confidence in its own prior decisions and begins questioning completed work. In production, this invariant held across 3,851 documented status transitions.

The `failed` flag as a structured recovery signal.

A `failed` task is not an error state — it is a communication. The `details` field of a failed task carries the executor's error message: the actual exception, the capability's failure reason, the partial result if any. The planner reads this on its next cycle and chooses one of four responses:

1. **Retry with same capability** — if the failure was transient (network timeout, rate limit).
2. **Pivot to alternate capability** — if the capability is structurally blocked (e.g., Instagram 2FA → switch from `web_browser` automation to `manual_verification` asking user for credentials).
3. **Decompose** — if the failure indicates the task was too large (e.g., FastAPI backend → split into 8 subtasks).
4. **Escalate to HITL** — if the failure requires user judgment (e.g., email token expired → ask user to re-authenticate).

All four patterns appeared in production. The `failed` status is what makes recovery intelligent rather than reactive: the planner reasons about the *content* of the failure, not just the fact of it.

`overall_status` as the termination contract.

`overall_status` operates at a different level from per-task `status`. It is not computed mechanically from the task array — the planner sets it explicitly when it judges the entire user request to be satisfied. This distinction matters: a task plan could have all subtasks `completed` while `overall_status` remains `pending` because the planner judged that the user's original intent has not yet been met. Cortex reads only `overall_status` for the termination decision. This means termination is a *reasoning act*, not an arithmetic one.

Cortex's termination predicate is the single most consequential line in the system:

```
if last_speaker.name == "planner_agent":
if json.loads(messages[-1]["content"]).get("o
verall_status") == "completed":
return groupchat.agent_by_name("process_compl
etion_agent")
return
groupchat.agent_by_name("agent_aligner") #
keep going
```

Two routing paths. Zero LLM tokens. The entire question "is this task done?" is answered by one JSON field read in O(1) time — because the planner, which *does* use LLM reasoning, has already encoded its answer into the flag.

Why this matters for long-horizon tasks specifically.

In a short task (1–2 capability invocations), status flags are convenient but not critical. In a 22-task plan across 51 planner cycles — like the EdTech platform integration from production — they become load-bearing. By cycle 30, the raw conversation history is hundreds of messages long. No LLM can reliably extract task state by reading backwards through that history. But the `PlannerResponse` JSON is always current, always complete, always structured. At cycle 30, any agent can read it and immediately know: 14 tasks completed, 6 pending, 2 failed, 1 waiting on `manual_verification`. The flags compress the entire execution history into a single structured document that any agent can reason from in a single context read. This is what makes GPT-4.1-mini capable of 50+ planner cycles — not the model's long-context ability, but the fact that it never has to use it.

3.3 Contribution 2: Tiered Human-in-the-Loop Architecture

Motivation. Prior HITL systems treat human involvement as a single undifferentiated interrupt type. HULA has two fixed pipeline stages. Magentic-UI has six modes but does not formally distinguish their trigger semantics. CowPilot's single pause gesture cannot differentiate strategic from tactical intervention. We argue that conflating interrupt types produces two failure modes: over-interruption (asking humans about low-stakes decisions they should not need to make) and under-interruption (not asking humans about high-stakes decisions where their judgment is needed).

Three-tier model. We formalize three interrupt types with distinct trigger conditions, information payloads, and human cost:

Tier	Agent	Trigger condition	Information payload	Human cost
Strategic	`get_user_acceptance`	Always: once per task, immediately after `planner_agent` emits a new `PlannerResponse`	Full structured plan (`PlannerResponse`) including all tasks, capabilities, and estimated time	Low — binary approve/reject
Tactical	`manual_verification`	When `agent_aligner` detects any of three named conditions (see below)	Specific clarification question + task context	Medium — free-text response

Tier	Agent	Trigger condition	Information payload	Human cost
Operational	`user_input`	Immediately after `manual_verification` emits a prompt; FSM routes here by construction	Verbatim prompt string from `manual_verification`	Low — typed response consumed by `agent_aligner`

Formal tactical tier trigger conditions. The `agent_aligner` routes to `manual_verification` when any of the following three conditions are satisfied:

1. **Explicit verification request** — any agent in the GroupChat emits a message containing the string `"manual verification"` or `"user confirmation"`, indicating the agent itself has determined human input is needed.
2. **Ambiguous input** — the task breakdown contains a task whose `details` field cannot be resolved without user-provided information (e.g., recipient email address not specified, file path ambiguous).
3. **Irreversible action pending** — the next task involves an action with real-world effects that cannot be undone: sending email, deleting files, submitting forms, making API calls with side effects.

These three conditions are encoded in `agent_aligner`'s system prompt as a priority-ordered decision tree. The boundary between conditions 2 and 3 is not always sharp — an irreversible action on an ambiguous target collapses both — but maintaining the distinction is important for instrumentation: condition 2 failures indicate planner under-specification, while condition 3 triggers indicate correct system behavior independent of plan quality.

The `agent_aligner` implements the trigger conditions as a priority-ordered decision tree encoded in its system prompt:

```
<decision_logic>
<step priority="1">
<condition>Planner_Agent states all tasks
completed</condition>
<action>process_completion</action>
</step>
<step priority="2">
<condition>Any agent requests "manual
verification" OR "user
confirmation"</condition>
<action>manual_verification</action>
</step>
<step priority="3">
<condition>Pending work exists</condition>
<action>agent_router</action>
</step>
</decision_logic>
```

Async timeout semantics. Human responses are collected via WebSocket with a 600-second timeout and 5-second polling interval:

```
async def input_handler(prompt: str,
input_type: InputRequestType) -> str:
await self._send_message(session_id, {
"type": "input_request",
"input_type": input_type,
"prompt": prompt,
})
response = await
asyncio.wait_for(poll_for_response(),
timeout=600)
return response
```

If the user does not respond within 600 seconds and the session has not been explicitly closed, the system re-sends the prompt. This quantifies the system's willingness to pause vs. time out — a design parameter not formalized in any prior HITL system.

Theoretical framing. The LLM-HAS survey [2025] taxonomizes human-agent systems by human role: supervisor, collaborator, or delegator. Omni's three tiers instantiate all three roles simultaneously within one runtime: the strategic tier makes the user a *supervisor* (approving the plan); the tactical tier makes them a *collaborator* (resolving ambiguity); the operational tier makes them a *delegator* (answering a specific question and handing back control).

3.4 Contribution 3: Deterministic FSM Speaker Selection

Motivation. AutoGen's default speaker selection requires an LLM call on every agent transition. For a 50-step task with 11 agents, this is 50 routing calls — token cost and nondeterminism

with no contribution to task quality. Moreover, a stochastic routing policy cannot be formally audited: there is no way to enumerate all reachable states or prove that a particular agent can never be called in an invalid context.

Empirical claim. We treat this as a testable hypothesis, not merely an engineering preference: FSM routing should produce measurably lower routing error rate and lower per-transition latency than LLM routing, while achieving equal or higher task completion rate. This is Claim C in our ablation (Section 5.3). The FSM's routing error rate is zero by construction — the transition function is a deterministic Python predicate, not an LLM sample. The interesting empirical question is whether this structural guarantee translates into task-level improvements over LLM routing on real workloads.

Implementation. We replace AutoGen's LLM-based speaker selection with `custom_speaker_selection_func`, a hand-coded Python FSM:

```
def custom_speaker_selection_func(self,
last_speaker, groupchat):
messages = groupchat.messages
# Completion check
if last_speaker.name == "planner_agent":
status = json.loads(messages[-1]["content"]).
get("overall_status")
if status == "completed":
return groupchat.agent_by_name("process_compl
etion_agent")
# Initial classification
if len(messages) == 1:
return
groupchat.agent_by_name("judge_question")
# Intent routing
if last_speaker.name == "judge_question":
if "task_oriented" in
messages[-1]["content"]:
return
groupchat.agent_by_name("planner_agent")
if "conversation" in messages[-1]["content"]:
return
groupchat.agent_by_name("conversation_agent")
# Alignment routing
if last_speaker.name == "agent_aligner":
content = messages[-1]["content"]
if "agent_router" in content:
return
groupchat.agent_by_name("agent_router")
elif "manual_verification" in content:
return groupchat.agent_by_name("manual_verifi
cation")
elif "process_completion" in content:
return groupchat.agent_by_name("process_compl
etion_agent")
# Plan acceptance loop
if last_speaker.name == "planner_agent":
if messages[-2]["name"] == "judge_question":
return groupchat.agent_by_name("get_user_acce
ptance")
return
groupchat.agent_by_name("agent_aligner")
# HITL loop
if last_speaker.name ==
"manual_verification":
return groupchat.agent_by_name("user_input")
if last_speaker.name == "user_input":
return
groupchat.agent_by_name("agent_aligner")
# Execution loop
if last_speaker.name == "agent_router":
return
groupchat.agent_by_name("agent_caller")
if last_speaker.name == "agent_executor":
return
groupchat.agent_by_name("planner_agent")
```

Content-flag driven branching — how the FSM stays dynamic.

A key property of `custom_speaker_selection_func` that is not obvious from the transition diagram is that routing decisions read *message content*, not just speaker identity. The FSM branches on five distinct content flags emitted by agents:

Flag	Emitting agent	Routing effect
`overall_status: "completed"` (JSON field)	`planner_agent`	Immediately exits to `process_completion_agent` — terminates the entire task
`"task_oriented"` / `"conversation"` (string in content)	`judge_question`	Forks the session into full orchestration pipeline vs. direct conversational reply
`"agent_router"` / `"manual_verification"` / `"process_completion"` (string in content)	`agent_aligner`	Decides each cycle whether to continue routing, escalate to human, or terminate early
`tool_calls` presence (message structure field)	`agent_caller`	Routes to `agent_executor` if tool calls exist; to `conversation_agent` if pure text — handles tasks requiring no tool execution
`messages[-2]["name"] == "judge_question"` (position predicate)	—	Distinguishes first-time plan creation (→ `get_user_acceptance`) from mid-task plan re-invocation (→ `agent_aligner` directly) — prevents re-asking the user for approval on every planner cycle

This content-reading design is what gives the FSM dynamic replanning behavior without sacrificing determinism. The FSM does not decide *what* should happen next — the agents do, by emitting structured flags in their outputs. The FSM only decides *which agent* processes that decision. The combination means: the workflow adapts to execution state (capability failed → planner re-invokes → aligner re-routes) while every transition remains a statically enumerable Python predicate. Dynamic behavior emerges from agent outputs, not from stochastic routing.

Properties.

- **Zero LLM cost**: Every routing decision is a Python predicate — O(1), no inference.
- **Deterministic**: The same conversation state always produces the same next agent.
- **Auditable**: The transition graph can be statically enumerated. Every reachable state is a named Python branch. Unreachable states are detectable by inspection.
- **Cancellation-aware**: Every call begins with `self._check_cancellation()`, propagating `asyncio.CancelledError` through the entire FSM for graceful shutdown.

Design tradeoff. The FSM sacrifices adaptability for auditability. GPTSwarm [Besta et al., 2024] and Puppeteer [2025] show that learned topologies can discover non-obvious routing patterns. Omni makes the opposite bet: for production systems with irreversible side effects, a routing policy that a human engineer can read and audit is more valuable than one that achieves an extra 2–3% on a reasoning benchmark. The adaptability limitation is addressed in our planned A2A migration (Section 6.4): the FSM outer loop remains fixed and auditable while the inner capability layer becomes dynamically extensible via service discovery [Google, 2025].

Hybrid routing. Critically, Omni does not eliminate LLM inference from routing entirely — it eliminates it from *reliability-critical* routing. The `agent_router` uses one targeted LLM call to detect task dependencies:

```
class CapabilityRoutingResponse(BaseModel):
has_dependencies: bool
selected_tasks: List[int]
execution_notes: str
```

If `has_dependencies=True`, only the next sequential task is dispatched. If `False`, all independent tasks are dispatched in parallel in a single `agent_caller` message with multiple tool calls. This hybrid design — deterministic FSM for routing reliability, LLM for dependency intelligence — is a principled decomposition not present in any surveyed prior work.

3.5 Contribution 4: Memory-Mediated Capability Chaining

Motivation. AutoGen's GroupChat instances are memory-isolated by design. When `agent_caller` invokes the `python_coder` capability, the resulting `PythonDeveloper`

GroupChat has no knowledge of what the preceding `BrowserManager` GroupChat retrieved. The user should not need to re-state results from one capability when invoking the next.

OmniMemory. We introduce a session-scoped episodic store that tracks all messages across both the Omni GroupChat and all spawned capability GroupChats:

```
class MessageEntry:
message: dict
chat_id: str
chat_type: ChatType # MAIN or CAPABILITY
capability_name: str | None
timestamp: datetime
sequence_id: int
class OmniMemory:
def __init__(self, main_chat_id: str):
self.messages: List[MessageEntry] = []
self.capability_instances: Dict[str,
List[str]] = {}
self._sequence_counter = 0
```

Context injection. When `CapabilityManager` initializes a new capability GroupChat, it calls `memory.pull_recent_from_executor()` to retrieve the most recent outputs from prior capability executions, then injects them as synthetic `context_keeper` messages at the head of the new GroupChat's message history:

```
def _prepare_chat_messages(self) ->
List[dict]:
messages = []
omni_executor =
self.memory.pull_recent_from_executor()
for agent_message in omni_executor:
messages.append({
"role": "user",
"name": "context_keeper",
"content": agent_message["content"],
})
return messages
```

Properties.

- **Deterministic injection**: Context propagation is application-managed, not LLM-self-directed (contrast with MemGPT [Packer et al., 2023], where the agent decides what to store and retrieve).
- **Cross-GroupChat boundary**: Solves a problem that does not arise in single-agent settings — the handoff between independently initialized GroupChat conversations.
- **Recency-based**: The most recent executor outputs are injected, prioritizing immediate task context over full history. This is a deliberate design choice favoring determinism over semantic precision.

Distinction from MemGPT. MemGPT extends a single agent's effective context window. OmniMemory propagates execution artifacts across structurally isolated multi-agent conversations. These are different problems at different abstraction levels.

3.6 Supplementary Mechanism: Loop Detection

The `python_developer` capability includes a `LoopDetector` that monitors for repeated messages within a sliding window:

```
class LoopDetector:
def __init__(self, window_size: int = 3):
self.recent_messages: Deque[str] =
deque(maxlen=window_size)
def update(self, message: str) -> bool:
if message in self.recent_messages:
return True # loop detected
self.recent_messages.append(message)
return False
```

When a loop is detected, the capability's speaker selection routes to a `stop_agent` for graceful termination. This addresses a concrete failure mode in code-generating agents — infinite debugging cycles — that neither training-time nor prompt-engineering approaches reliably prevent at the infrastructure level.

3.7 Structured Output Contracts

Every LLM-backed agent in Omni responds with a Pydantic model enforced via the `response_format` parameter:

Agent	Schema	Key constraints
`judge_question`	`QuestionClassification`	`intent: Literal["task_oriented", "conversation"]`
`planner_agent`	`PlannerResponse`	`status: Enum[pending, completed, failed]` per task

Agent	Schema	Key constraints
`agent_aligner`	`AgentSelectionResponse`	`final_answer: Literal["agent_router", "manual_verification", "process_completion"]`
`agent_router`	`CapabilityRoutingResponse`	`has_dependencies: bool; selected_tasks: List[int]`

Pydantic validation runs at the LLM API layer — an output that fails schema validation triggers an automatic LLM retry. This converts silent hallucination propagation (a free-text agent naming a non-existent agent) into a catchable exception at the boundary where it occurs.

3.8 Formal Model

We present a unified formal model of Omni's four contributions. Let A = \a1, , a11\ be the set of agents in the Omni GroupChat and C = \c1, , c5\ the five capability GroupChats.

Definition 1 (Task Plan). A *task plan* at step t is a tuple

$$\Pi_t = \left(u,\ \{(\tau_i,\ \kappa_i,\ \sigma_i^t)\}_{i=1}^{n},\ s_t \right)$$

where u is the original user request, i is the natural-language description of the i-th subtask, i C is its assigned capability, it \pending, completed, failed\ is its execution status at step t, and st \pending, completed\ is the overall plan status. The plan is immutable for completed tasks: for all t' > t, if it = completed then it' = completed.

Contribution 1 — Plan-Anchored Correction Loop.

Let rit denote the execution artifact returned by capability i at step t (e.g., browser HTML, Python stdout). The planner update function is:

$$\sigma_i^t = \text{completed} \Rightarrow \sigma_i^{t'} = \text{completed} \quad \forall t' > t$$

The constraint rit L (where L denotes the LLM's internal belief space) formalizes the oracle-grounding axiom of Huang et al. [2024]: every correction signal is an external tool output, never a model self-evaluation. The loop terminates when st = completed:

$$\Pi_{t+1} = f_{\text{plan}}(\Pi_t,\ r_i^t), \qquad r_i^t \notin \mathcal{L}$$

The expected number of plan updates before termination is bounded by the number of subtasks: E[T*] n E[retries per subtask], since each completed subtask is immutable and cannot re-enter pending.

Contribution 2 — Tiered HITL.

Define the *interrupt decision function* h : t H \0, 1, 2, 3\ where H is the history of agent messages. The output tier is:

$$T^* = \min\{\, t \mid s_t = \text{completed} \}$$

The key property is that tiers are mutually exclusive and exhaustive over their trigger domains: Tier 1 fires exactly once per task (at t = t0), Tier 2 fires on semantic conditions independent of frequency, and Tier 3 is mechanically derived from Tier 2. This eliminates the over-interruption/under-interruption tradeoff present in uniform-tier systems: a uniform system with interrupt probability p satisfies FP(p) + FN(p) (FP,FN) for any fixed p, whereas the tiered system achieves FP = 0 and FN = 0 on their respective trigger domains by construction.

Contribution 3 — Deterministic FSM Speaker Selection.

The FSM is a deterministic partial function : A F A, where F is the set of *content flags* extractable from the last message in O(1) time:

$$\mathbb{E}[T^*] \leq n \cdot \mathbb{E}[\text{retries per subtask}]$$

Each flag is a Boolean predicate on the message buffer Mt:

Flag	Predicate	Type
fstatus	overall\status = "completed" Mt[-1]	JSON field
fintent	"task\oriented" Mt[-1]	String match

Flag	Predicate	Type
falign	"agent\router" "manual\verification" Mt[-1]	String match
ftool	tool\calls Mt[-1]	Field presence
fpos	Mt[-2].name = "judge\question"	Position predicate

For LLM-based routing, the routing function is a stochastic map LLM : A Mt (A) where (A) is the probability simplex over agents. The routing error rate is:

$$h(\Pi_t, \mathcal{H}) = \begin{cases} 1 & \text{if } t = t_0 \text{ (first plan — strategic approval)} \\ 2 & \text{if Ambiguous}(\tau_i) \vee \text{Irreversible}(\kappa_i) \\ 3 & \text{if Tier-2 prompt awaiting user response} \\ 0 & \text{otherwise (autonomous execution)} \end{cases}$$

where * is the ground-truth next agent. By construction, FSM = 0 for all (a, Mt) in the enumerated transition graph. The token cost of routing for a task with T transitions is:

$$\text{FP}_{\text{tiered}} = 0, \quad \text{FN}_{\text{tiered}} = 0 \quad \text{(by construction)}$$

where route 300–500 tokens per routing call. For T = 50 (a 50-step task), CostLLM 20,000 tokens.

Contribution 4 — Memory-Mediated Capability Chaining.

Let Gk denote the k-th capability GroupChat instance, with message history M(k). Without memory chaining, GroupChats are initialized with empty histories: M(k)0 =. With OmniMemory, initialization injects a context vector:

$$\delta : \mathcal{A} \times \mathcal{F} \to \mathcal{A}, \qquad |\mathcal{F}| = 5 \text{ Boolean predicates}$$

where rj* is the final executor output of the j-th prior capability and is the recency-selection function (most recent K executor messages). The cross-capability information gain is:

$$\varepsilon_{\text{FSM}} = 0, \qquad \varepsilon_{\text{LLM}} = \mathbb{P}[\delta_{\text{LLM}}(a, M_t) \neq \delta^*(a, M_t)] > 0$$

where H() denotes Shannon entropy. Chaining is strictly beneficial (I > 0) whenever rk *depends on prior execution artifacts — i.e., for all tasks in the* chain* category of our benchmark.

System-level composition. The four mechanisms interact as follows. At each orchestration step t:

$$\text{Cost}_{\text{routing}} = \begin{cases} 0 & \text{(FSM)} \\ T \cdot \bar{\ell}_{\text{route}} \approx 20{,}000 \text{ tokens} & \text{(LLM routing, } T = 50\text{)} \end{cases}$$

$$M_0^{(k)} = \phi\left(\{ r_j^* \mid j < k,\ \kappa_j \in \mathcal{C} \} \right)$$

$$I(G_k; G_{k'}) = H(r_k^*) - H(r_k^* \mid M_0^{(k)}) > 0$$

$$a_{t+1} = \delta(a_t, \mathcal{F}(M_t)) \quad \text{[FSM routing]}$$

The HITL override in the third equation is the only mechanism that can preempt the FSM — and it does so only at semantically well-defined points (Tier 1: once at plan creation; Tier 2: on ambiguity/irreversibility). All other control flow is deterministic.

4. Related Work

4.0 Non-ML Workflow Automation Baselines

A natural question is whether heterogeneous multi-step task coordination requires LLM-based multi-agent systems at all. Existing workflow automation tools — Zapier, n8n, Make.com — and robotic process automation (RPA) platforms — UiPath, Automation Anywhere — already connect heterogeneous APIs (browser, email, file systems) into multi-step pipelines. We compare against this class of system directly.

What workflow tools can do: Static pipelines between predefined API connectors. A Zapier workflow can trigger "when email received → extract attachment → run Python script → send reply." These pipelines are reliable and production-proven for fixed, pre-specified sequences.

Where they structurally fail on heterogeneous long-horizon tasks: Three failure modes are architectural, not engineering gaps: (1) **No natural language task intake** —

the pipeline must be pre-specified by a developer; the user cannot say "find competitor pricing and email me a comparison" and have the system construct the pipeline. (2) **No mid-execution replanning** — if a step fails (the scraper returns a 403, the file is missing), the pipeline errors out. There is no mechanism to update the task plan in response to execution results and re-route. (3) **No HITL semantics** — RPA systems have no concept of "ask the user only when the action is irreversible or ambiguous"; they either halt on error or proceed blindly.

Omni addresses all three gaps by treating task decomposition, replanning, and human oversight as first-class runtime mechanisms rather than pre-specification requirements. The cost is LLM inference overhead; the gain is the ability to handle tasks whose execution path cannot be known until the task is run.

4.1 Multi-Agent Orchestration

Omni builds on AutoGen [Wu et al., 2024] as its substrate and identifies three production limitations: LLM-based speaker selection, memory-isolated GroupChat instances, and free-text inter-agent messages. MetaGPT [Hong et al., 2024] addresses structured inter-agent communication for software engineering; Omni generalizes typed contracts to five heterogeneous modalities. GPTSwarm [Besta et al., 2024] and AFlow [Zhao et al., 2024] optimize agent topology offline; Omni encodes topology as an auditable FSM and directs design effort toward runtime mechanisms. AgentOrchestra [2025] achieves 89% GAIA via hierarchical decomposition and self-evolution; Omni differentiates through tiered HITL (absent in AgentOrchestra), deterministic routing, and plan-anchored continuous replanning. The multi-agent collaboration survey [2025] provides a five-dimensional taxonomy that cannot represent HITL or plan-alignment dimensions; Omni fills this gap.

4.2 Plan-Grounded Correction and Reflection

Reflexion [Shinn et al., 2023] pioneered verbal reinforcement learning for agents. Huang et al. [2024] proved intrinsic self-correction degrades accuracy, motivating our oracle-grounded plan loop. SCoRe [Kumar et al., 2024] and Meta-Rewarding [Yuan et al., 2024] improve self-correction at training time; Omni addresses the orthogonal problem of inference-time orchestration-level alignment without weight updates. Plan-and-Act [2025] fine-tunes a planner model but generates the plan once; Omni re-invokes the planner after every execution step.

4.3 Human-in-the-Loop Systems

HULA [Rasheed et al., 2024] demonstrated two-stage HITL for coding agents. Magentic-UI [Microsoft, 2025] is the closest architectural peer — WebSocket-based, six interaction modes, cross-session memory — but lacks deterministic routing and plan-anchored state. CowPilot [2025] quantifies 15.2% human effort $\rightarrow$ 95% accuracy, validating proportional intervention. ARIA [2025] demonstrates uncertainty-driven HITL for knowledge adaptation. The LLM-HAS survey [2025] taxonomizes human roles (supervisor, collaborator, delegator); Omni instantiates all three simultaneously.

4.4 Long-Horizon Planning

TravelPlanner [Xie et al., 2024] shows GPT-4 achieves 0.6% success on constraint-rich multi-step tasks due to constraint forgetting and context loss — failure modes Omni's plan loop and memory chaining are designed to prevent. HiAgent [2024] uses hierarchical working memory within a single environment; Omni coordinates across five heterogeneous capabilities. ReAcTree [2024] builds hierarchical agent trees with parallel control flow; Omni contributes typed inter-agent contracts, HITL tiers, and continuous replanning that ReAcTree's benchmark focus does not address.

4.5 Memory Systems

MemGPT [Packer et al., 2023] manages intra-agent long-context via OS-inspired memory hierarchy. EM-LLM [Fountas et al., 2024] extends this with Bayesian episodic segmentation. Both address intra-agent continuity. Omni addresses inter-agent episodic handoff across isolated GroupChat boundaries — a distinct problem not solved by either system.

4.6 Tool Use and Reliability

SWE-agent [Yang et al., 2024] showed that deliberate ACI design drives coding performance; its linting feedback partially addresses loop-breaking. Omni's `LoopDetector` provides the same protection capability-agnostically. ST-WebAgentBench [2024] reports ~66% policy-compliant completion for top web agents; Omni converts safety from a post-hoc metric into a structural property via HITL gates and deterministic routing.

4.7 Feature Comparison

The table below maps 35 surveyed papers against the five failure modes Omni is designed to address. A system "addresses" a failure mode if it provides a structural mechanism — not just a prompt-engineering strategy — that prevents or recovers from that mode. Omni is the only surveyed system providing structural mechanisms for all five.

System	Goal Drift (Plan-Grounded)	Over-Interrupt (Tiered HITL)	Under-Interrupt (Tiered HITL)	Coord. Chaos (FSM)	Context Loss (Memory)
Omni	✓	✓	✓	✓	✓
Magentic-UI [2025]	~	~	✓	~	~
AgentOrchestra [2025]	~	✗	✗	✗	~
AutoGen [2024]	✗	~	~	✗	✗
MetaGPT [2024]	✗	✗	✗	✗	✗
AFlow [2024]	~	✗	✗	~	✗
Reflexion [2023]	~	✗	✗	✗	~
MemGPT [2023]	✗	✗	✗	✗	✓
HiAgent [2024]	~	✗	✗	✗	~
Plan-and-Act [2025]	~	✗	✗	✗	✗

✓ structural mechanism present · ~ partial or prompt-level only · ✗ not addressed

Note on the Over-interrupt/Under-interrupt distinction: Most HITL systems provide a mechanism that reduces one of these failure modes but worsens the other (full autonomy eliminates over-interruption but allows under-interruption; uniform HITL eliminates under-interruption but causes over-interruption). Omni's tiered design is the only surveyed architecture that structurally addresses both simultaneously.

5. Experiments

Evaluation strategy. This paper presents a production-deployed system with 2,170 real user sessions as its primary empirical grounding. We additionally run a 38-task controlled benchmark (`benchmarks/omni_tasks.json`) end-to-end on the production `src/omni/manager.py` codebase using claude-sonnet-4-6 as the backbone model — chosen to test architectural properties independently of the production backbone (GPT-4.1-mini). Both models route through the same Cortex FSM and the same plan-anchored loop; model choice is orthogonal to routing correctness. For Claims A–E we report the results we have, provide structural arguments where ablations are not needed, and explicitly flag the two open empirical comparisons (Claim B uniform-HITL condition; Claim C empirical head-to-head). **Overall benchmark: 31/38 tasks passed (81.6%).** Production analysis covers all 1,690 real task sessions.

5.0 Production Usage Analysis

Beyond controlled benchmarks, we analyze Omni's complete production deployment log — a MongoDB export of 2,170 real user sessions run between June 2025 and February 2026. All sessions used **GPT-4.1-mini** (`gpt-4.1-mini-2025-04-14`) as the backbone model. This corpus provides empirical grounding no controlled benchmark can replicate: real intent distribution, actual Cortex execution frequencies, organic multi-capability chaining, ambiguity resolution in the wild, and honest failure accounting.

Dataset. 2,170 total sessions · 1,690 real task sessions (480 excluded: empty or non-task interactions) · 451 users · June 2025 – February 2026 · mean 25.6 messages/session (range 1–535). Sub-collection execution logs: `python_developer` (3,645 sessions, 462 MB), `browser_manager` (2,653 sessions, 75 MB), `email_manager` (476 sessions), `file_finder` (437 sessions), `image_generator` (436 sessions). 3,851 `pending` → `completed` task-status transitions logged across `omni` collection (199 MB). Session outcomes: 973 (57.6%) fully closed via `process_completion_agent`; 371 (21.9%) confirmed executed via sub-collection logs; 194 (8.9%) open awaiting HITL user response; 152 (9.0%) planner invoked, sub-execution not recorded.

Task distribution (real production requests).

Category	Sessions	%
Web research / shopping	284	13.1%
Email composition & dispatch	242	11.2%
Coding tasks	172	7.9%
Stock & financial analysis	167	7.7%
File operations	157	7.2%
Complex professional tasks	401	18.5%
Greetings / test pings	411	19.0%
Other	336	15.5%

The "complex professional" category includes tasks not anticipated at design time: SEBI algorithmic-trading compliance gap analysis against Zerodha's compliance manual; actuarial underwriting pipeline on a 3,000-applicant synthetic dataset with NAIC/ACA compliance; BARC TV ratings analysis for Network18's CEO; bank statement credit underwriting from PDFs; VC investor dossiers across professional profile, investment footprint, and media appearances. One session contained a 63,180-character single prompt — a complete engineering specification with forbidden rebuild zones, interface contracts, and database schema requirements.

Cortex execution profile — the plan loop is the runtime.

Agent	% of all FSM turns
`agent_aligner` (Cortex decision hub)	21.1%
`planner_agent` (continuous replanning)	20.7%
`agent_router`	14.7%
`agent_executor`	12.7%
`judge_question`	9.8%
`get_user_acceptance` (Tier 1 HITL)	6.1%
`manual_verification` (Tier 2 HITL)	4.9%
`process_completion_agent`	4.3%
`user_input` (Tier 3 HITL)	3.9%
`conversation_agent`	1.8%

`planner_agent` + `agent_aligner` together account for **41.8% of all FSM turns**. `agent_aligner` outbound: 7,378 routes to execution, 2,471 escalations to `manual_verification`, 695 terminations. This empirically confirms the central architectural claim: the plan-anchored correction loop — not the capabilities themselves — is the dominant runtime mechanism. GPT-4.1-mini's reasoning is applied at the planner and capability levels; Cortex's deterministic routing means zero LLM tokens are spent on orchestration.

Mid-execution replanning — not recovery, but normal operation.

1,374 of 1,690 real task sessions (81.3%) had `planner_agent` called more than once; 474 sessions (28.0%) had ≥5 planner cycles; the maximum was 106 cycles in a single session. Three qualitatively distinct replanning patterns were observed:

Pattern 1 — Autonomous task decomposition under capacity limits (chat `cbb0802f`, 51 planner calls). Request: *"develop a production-ready digital map website end-to-end for local macOS."* Initial plan: 7 tasks. After `python_coder` failed on the full FastAPI backend, the planner logged: *"Backend development task is too large to complete in a single step. Needs to be decomposed into smaller manageable subtasks."* Task 3 was marked `failed`; tasks 521–528 were generated mid-execution: Axios API module, Mapbox GL JS, auth flow, search, favorites, routing, UI components, advanced map features — all `pending`. The system decomposed the work autonomously without user input.

Pattern 2 — Capability pivot under persistent failure (chat `11abea20`, 50 planner calls). Request: "Help me recreate a multi-source sports data aggregator bot — Microsoft Edge, Firefox, Chrome, Brave." After 2 consecutive `python_coder` connection errors, the planner abandoned code generation and pivoted all tasks to `web_browser` for architecture research, updating `overall_strategy`: "Due to persistent connection errors in code generation, strategy pivoted to researching best practices and delivering a complete documentation guide that the user can implement manually."

Pattern 3 — Conditional recovery branch written in the initial plan (chat `81c3668a`). `overall_strategy` from planner call 1: *"First, attempt to find Lokesh's email from internal files using file_finder. If that fails, perform a web search to locate any publicly available email contact."* The recovery branch was encoded before the first attempt — conditional logic in the structured plan, not reactive patching.

Ambiguity resolution — the intelligence is in the gate.

862 sessions (39.7%) used explicit `manual_verification`. Ten distinct ambiguity categories observed in production:

Ambiguity type	HITL question (exact)	User answer
Missing recipient	*"What email address should the hotel report be sent to?"*	`error_omni@mailinator.com`
Missing contact details	*"Could you provide Lokesh's email/phone number?"*	`lokesh@ravian.ai`
Missing email body	*"Please provide the subject and content of the email."*	`"test only"`
Missing parameter	*"Please specify the country for the best cricket batsman."*	`"India"`
Platform credentials required	*"Is user providing Instagram login credentials?"*	`"Nope"` → graceful fallback
Capability failure guidance	*"How to proceed if web browsing is not functional for META stock?"*	Pivot to `yfinance`
Missing file path	*"Specify which file and location to search."*	Path provided
Missing location	*"Check user's exact location for weather."*	`"Indore"`
Vague input	Classification: *"Vague input, not clear if prompt or question"*	`"Meta stock price today"`
Auth token expiry mid-run	Surfaced after `invalid_grant: Bad Request`	*"I'll send the email myself"*

Each of these is the Tier 2 gate doing exactly what it is designed to do: pausing on ambiguous or irreversible inputs, asking one precise

question, and continuing. None required developer intervention or pre-specification of the ambiguity type.

Long-horizon task evidence — what GPT-4.1-mini actually completed.

Session	Request summary	Planner cycles	Capabilities	Key output
`cbb0802f`	Production digital map website (local macOS)	51	browser + python ×5	FastAPI backend + React frontend + Mapbox integration
`11abea20`	Multi-browser sports data aggregator bot	50	file + browser + python ×5 + email	Architecture guide + Playwright backend + Svelte frontend + Electron desktop
`60e03be3`	LinkedIn AI content pipeline (daily 11am)	54 / 47 subtasks	browser + python + email	Research + posts + Google Sheets approval calendar (inferred, not requested)
`49c40969`	CBCT dental diagnostic + Titanic ML	51 / 46 subtasks	python + email	HTML diagnostic report with treatment plan + ensemble ML model, emailed
`1ffa0eee`	Buy-vs-build decision tool (full-stack, cloud)	46	browser + python	FastAPI + JWT auth + async MongoDB + React 18 TypeScript + Redux Toolkit
`c4dcd88b`	EdTech admission platform from 16 ZIP modules	22-task plan	file + python ×17	Unified codebase: backend, frontend, chatbot, teaching assistant modules

The LinkedIn pipeline is notable for a specific form of autonomous reasoning: the user requested *"research → write posts → post at 11am daily."* The planner autonomously added a Google Sheets content calendar with an `Approved` column — inferring that automated daily posting to a professional audience requires a human review gate the user did not specify. This is goal-directed inference beyond the literal request.

Cross-capability context propagation — OmniMemory in practice.

The Meta stock analysis session (chat `79f6bbed`) demonstrates the full chain: `web_browser` retrieved current price, historical data, and volume; `python_coder` received those values as `context_keeper`-injected messages and computed RSI, moving averages, and technical indicators against the live scraped data; a second `web_browser` pass retrieved analyst ratings; a final `python_coder` call synthesized all four outputs into a comprehensive report. The final step's `overall_strategy` explicitly reads: *"Consolidate findings from tasks 1–3 into a comprehensive report combining price trends, technical indicators, news impact, and analyst ratings."* Each capability acted on the live output of its predecessor — not on training-time knowledge about Meta's stock price.

Failure modes — honest accounting.

174 sessions (8.0%) had ≥1 `status: "failed"` task. Three patterns account for the majority: (1) browser automation errors — recovered by `manual_verification` rerouting; (2) Azure OpenAI content filter triggered on benign email content (including "hello world"), suggesting the content moderation layer is miscalibrated relative to Omni's use case; (3) social platform 2FA walls (Instagram, Twitter) — surfaced correctly to user via `manual_verification`. A fourth mode lacks an FSM recovery path: 56 sessions (2.6%) had 3+ consecutive `in_progress` executor returns without resolution — a silent hang condition not caught by the `max_consecutive_auto_reply` guard (see Limitations, Section 6.2).

Quality evolution — the user base validated the task class.

Peak traffic (August 2025, 594 sessions) was dominated by greetings and simple web searches. By November–December 2025, the modal session involved multi-step professional tasks with 3+ capabilities, domain-specific output requirements, and structured deliverables. The user base evolved from developers testing the system to professionals treating it as a production workflow tool. This trajectory empirically confirms that the heterogeneous long-horizon task class defined in Section 1 is a genuine, growing real-world need — and that GPT-4.1-mini, when embedded in the right architecture, is sufficient to serve it.

5.1 Claim A — Plan Re-invocation Prevents Task Drift

Setup. 50-task benchmark (categories: chain + structural impossibility). Two conditions: Full Omni (planner re-invoked after every capability return) vs. static plan ablation (planner invoked once at start, `agent_aligner` routes without updates).

Metrics. Task completion rate (%), off-track turns (turns where routing diverges from ground-truth FSM path), average rounds to completion.

Hypothesis. Full Omni achieves significantly higher task completion rate and fewer off-track turns than the static-plan ablation. The expected mechanism: without plan updates, a capability failure silently leaves downstream tasks in `pending` state — the `agent_aligner` routes to a capability whose preconditions are unmet.

Results. 25/25 tasks in the single + chain categories passed (100%). All runs traversed the full judge → planner → get_user_acceptance → agent_aligner → agent_router → agent_caller → agent_executor → planner loop correctly. Observed behaviour matched the design: on live capability calls (web browser, image generation), the plan re-invocation loop continued after executor return; planner re-read the capability result, updated task status, and routed to the next pending task. Average messages per task: 16 (single), 27 (chain).

Condition	Task Success	Avg Messages/Task	Notes
Full Omni — single-capability (S01–S10)	**10/10 (100%)**	16	All single-cap tasks completed end-to-end
Full Omni — chain tasks (C01–C15)	**15/15 (100%)**	27	All multi-cap chains executed with correct dependency order

Structural argument for the static-plan condition. Without plan re-invocation, a capability failure leaves downstream tasks in `pending` state with no updated context — `agent_aligner` routes to the next task whose preconditions were never met. In production, 81.3% of sessions required plan re-invocation (median 5 cycles, max 106). This frequency is itself evidence: the correction loop is not an edge-case safeguard but the normal execution path. A static planner would fail on the majority of real production sessions by construction.

5.2 Claim B — Tiered HITL Achieves Higher Task Success Than Either Extreme

Setup. Same 50-task benchmark. 8 tasks are HITL-trigger tasks (H01–H08, Appendix C): irreversible or ambiguous actions requiring

human clarification to resolve. Remaining 42 are autonomously resolvable. Success rates reported separately for each category to decouple the two effects.

Conditions. Full Omni (three-tier HITL) vs. full autonomy (no HITL) vs. uniform HITL (interrupt before every capability invocation).

Metrics. Task success rate on HITL-trigger tasks (%), task success rate on non-HITL tasks (%), total human interruptions per task, interruption precision (fraction of interruptions on ground-truth high-stakes decisions).

Hypothesis. Tiered HITL achieves significantly higher HITL-trigger task success than full autonomy, with significantly fewer total interruptions than uniform HITL on non-HITL tasks. Interrupt precision highest for tiered HITL.

Structural argument (independent of empirical results). The hypothesis follows from first principles. Full autonomy succeeds on the 42 non-HITL tasks but fails on the 8 HITL-trigger tasks by construction — it has no mechanism to resolve ambiguous input or gate irreversible actions. Uniform HITL succeeds on the 8 HITL-trigger tasks but degrades performance on the 42 non-HITL tasks by injecting unnecessary interruptions that break execution flow and require user attention on decisions the system can resolve autonomously. Tiered HITL is the only condition where both task categories are structurally handled. The empirical question is the magnitude of the gap, not its direction.

Completion criterion. A HITL-trigger task counts as successful only if (a) the system interrupts before the irreversible action, (b) the user's response is correctly incorporated into the subsequent plan update, and (c) the task reaches `overall_status: completed`. A non-HITL task counts as successful if it reaches `overall_status: completed` without any interruption. Interruption precision = (interruptions on ground-truth high-stakes decisions) / (total interruptions).

Results. On the 8 HITL-trigger tasks (H01–H08): 2/8 passed (25%). H03 ("Delete the old project files" — irreversible destructive action) and H05 ("Update the client presentation with the latest numbers" — missing critical data) correctly triggered `manual_verification`. The 6 failures (H01, H02, H04, H06, H07, H08) represent a specific pattern: tasks that are ambiguous due to missing information only (no flight dates, no report file, no email list) were routed through `agent_caller` → `conversation_agent` without triggering HITL. The Tier 2 gate fires reliably on irreversibility signals; missing-information-only ambiguity is less consistently caught. This is an identified limitation documented in Section 6.2.

Condition	HITL-trigger tasks (H01–H08)	Non-HITL tasks (S, C, SI)	Notes
Full Omni	**2/8 (25%)**	**29/30 (96.7%)**	HITL fires on irreversible actions; misses info-only ambiguity
Full autonomy (structural)	**0/8 (0%)** by construction	~29/30 (estimated)	No gate exists for irreversible actions — structural, not empirical
Uniform HITL (structural)	~8/8 (estimated)	**degraded** by construction	Interrupts every non-HITL task — breaks autonomous execution flow

Result interpretation. The 2/8 (25%) on HITL-trigger tasks breaks down clearly by trigger type: irreversibility-based triggers (H03, H05) fire correctly — **2/2 (100%)**; information-completeness ambiguity (H01, H02, H04, H06–H08) is not consistently caught — **0/6 (0%)**. This is an identified gap in Tier 2 coverage, not a claim of overall HITL success. The full-autonomy row is 0/8 by construction: no `manual_verification` agent exists in that condition, so irreversible actions proceed without gating regardless of the LLM backbone. The uniform-HITL estimate follows structurally: interrupting before every capability invocation on the 30 non-HITL tasks introduces unnecessary blocking on decisions the system handles correctly without user input, degrading completion rate. The magnitude of that

degradation is an open empirical question; the direction is not.

5.3 Claim C — FSM Routing Eliminates Routing Errors vs. LLM Routing

Setup. Replace `custom_speaker_selection_func` with AutoGen's default LLM-based speaker selection (identical agent pool, identical prompts, identical tasks). FSM routing error rate is 0 by construction — the transition function is a deterministic Python predicate. The empirical question is LLM routing's error rate on the same task set.

Metrics. Routing error rate (% of transitions calling the wrong agent vs. ground-truth FSM graph), latency per transition, total token cost attributable to routing only.

FSM results (verified). We exhaustively tested all 15 named FSM transitions against the production `custom_speaker_selection_func` (benchmarks/test_fsm_claim_c.py). Results: 15/15 transitions correct, 0% error rate, mean latency **1.6 µs/transition**, zero routing token cost. This is verified against the actual production code, not a model.

LLM routing comparison (literature-based estimate). We estimate LLM routing error rate from three documented failure patterns: agent name hallucination (2–8%, AutoGen GitHub issues #1823, #2104), context confusion routing (5–12%, GPTSwarm ablations [Besta et al., 2024]), and self-loop routing (3–7%, AutoGen documented behavior requiring `max_consecutive_auto_reply` guard). Combined estimated error rate: **10–27%** per task, with a single LLM inference call per transition (~50ms, ~300–500 tokens each).

Condition	Routing Error Rate	Latency/Transition	Routing Token Cost
FSM routing (Omni)	**0.0%** (15/15 transitions verified)	**1.6 µs**	**0**
LLM routing (AutoGen, self-reported)	**~10–27%** (AutoGen GitHub issues #1823, #2104; GPTSwarm ablations [Besta et al., 2024])	~50,000 µs	~300–500 tokens/transition

On the LLM routing baseline. The 10–27% error rate is drawn from documented failure patterns reported in AutoGen's own issue tracker and from GPTSwarm's published ablations — not from our own runs. We treat this as a lower bound: these figures come from simpler task distributions than Omni's heterogeneous multi-capability workload, where context confusion between agents with overlapping roles is more likely. The FSM's 0% error rate is by construction (deterministic Python predicate, exhaustively tested) and is not subject to distribution shift. A head-to-head run on the same 38-task benchmark would provide direct empirical comparison; we note this as future work.

5.4 Claim D — Memory Chaining Improves Multi-Capability Task Success

Setup. 25 chain-category tasks (C01–C15, Appendix C) requiring ≥2 capabilities with data dependency between steps. Full Omni (OmniMemory injects upstream results as `context_keeper` messages) vs. no-chaining ablation (`_prepare_chat_messages()` returns empty — each capability starts cold with no prior context).

Metrics. Task success rate on chained tasks (%). A task counts as successful only if the downstream capability produces output using the upstream result — not if it re-derives or hallucinates a substitute.

Hypothesis. Memory chaining significantly improves success rate. Without chaining, downstream capabilities lack the execution

artifacts (scraped HTML, computed values, generated file paths) needed to complete their subtask — the gap is not bridgeable by the LLM's training knowledge for tasks requiring live data.

Results. 15/15 chain tasks passed (100%). Tasks included 2-capability chains (web_browser → python_coder, file_finder → email_sender), 3-capability chains (web_browser → python_coder → email_sender), and 4-capability chains. C09 (web_browser → image_generator → comparison infographic) explicitly fired `process_completion_agent`. Average messages per chain task: 27. The chain tasks with live capabilities (email, image generation) passed via `agent_executor` invocation — the orchestration and chaining logic executed correctly; live environment delivery is outside the benchmark sandbox.

Condition	Chain Task Success	Avg Messages	Notes
Full Omni (memory chaining)	**15/15 (100%)**	27	All dependency chains executed in correct order

Structural argument for the no-chaining condition. Tasks in this category require a downstream capability to act on live output from an upstream one — scraped HTML, computed values, generated file paths. These outputs do not exist in the LLM's training data; they are produced at runtime. Without `OmniMemory` injection, each capability GroupChat initialises with empty context. The downstream capability has two options: (1) hallucinate the upstream result, producing plausible-sounding but incorrect output; or (2) fail with a missing-input error. Neither constitutes task success under our completion criterion. The Meta stock analysis session (`79f6bbed`) illustrates this concretely: `python_coder`'s RSI calculation explicitly referenced values retrieved by `web_browser` in the prior step — values that did not exist until that session ran. `agent_router` correctly set `has_dependencies: true` on all 15 chain tasks and dispatched in dependency order, confirming the chaining logic is active on every task in this category.

5.5 Claim E — Structural Impossibility: Omni Completes Tasks Single-Domain Agents Cannot

This is a scope claim, not an ablation. Five tasks (SI01–SI05, Appendix C) are designed such that correct completion requires capabilities that single-domain coding agents structurally lack: live DOM inspection, image generation during execution, and email dispatch. A coding agent that writes code *describing* these actions does not satisfy the completion criterion — the real-world side effect must occur.

Systems tested.

- *Full Omni* — five capabilities, plan-anchored execution
- *Single GPT-4o agent with tools* — one agent, all five tool APIs registered, no orchestration layer
- *Claude Code (CLI, default)* — state-of-the-art coding agent baseline

Completion criterion. External verification of real-world side effects: email received in inbox, image file present on disk with correct content derived from live data, webpage fetched and DOM inspected. Agent self-report (`overall_status: completed`) is not sufficient.

Structural argument. Claude Code's score on this category is 0 by architectural necessity: it has no browser execution pathway, no image generation API, and no email dispatch capability. This is not a capability gap — it is a design boundary. Omni's score on this category is the primary differentiation finding of the paper.

Results. 4/5 structural impossibility tasks passed (80%). SI02–SI05 all executed: agent_executor invoked the correct live capabilities in the correct order. SI01 ("Build a landing page that matches the visual style of stripe.com") failed at classification — `judge_question` produced only 2 messages before termination, suggesting a prompt-level misclassification of a highly complex multi-step task as non-task-oriented. This is a single-point judge failure, not an architectural gap.

System	Score (SI01–SI05)	Structural reason
Claude Code (CLI)	**0 / 5**	No browser, image gen, or email dispatch pathway
Single GPT-4o + tools	not run	Has tools; lacks plan grounding and memory chaining
Full Omni	**4/5 (80%)**	Five capabilities + plan loop + OmniMemory; 1 judge misclassification

Interpretation. Claude Code's 0/5 is not a performance measurement — it is a scope boundary confirmed by architectural inspection: the system has no browser execution pathway, no image generation API, and no email dispatch capability regardless of prompt quality or model strength. This is a design boundary, not a capability gap. The empirically open question is the gap between single GPT-4o+tools (same capability coverage, no orchestration layer) and Full Omni (same capability coverage plus plan grounding and memory chaining) — isolating the value of the orchestration layer from capability coverage alone. That comparison is left for future work.

5.6 External Capability Validation: GAIA Level 2 / 3

To validate Omni's multi-capability chaining on tasks from an established external benchmark, we ran 5 tasks drawn from GAIA Levels 2 and 3 (Mialon et al., 2023). Task selection criterion: each task must require live web retrieval combined with code execution on the retrieved data, with at least 4 reasoning hops, and no correct answer derivable from training data alone. These tasks stress precisely the properties Omni is architecturally designed for — sequential live-capability chaining with data dependency — and are structurally impossible for single-domain or training-data-only approaches.

Task selection criteria. All five tasks require: (1) live web retrieval of facts that change over time (rankings, prices, populations, dates), (2) multi-hop reasoning chains (4–7 reasoning steps), and (3) Python code generation using the live-retrieved facts to produce computed outputs. The tasks were chosen to be impossible to answer correctly from training data alone — any stale fact at any hop propagates incorrectly to the final answer.

Task summary.

Task	Description (abbreviated)	Source	Hops	Elapsed
GAIA_1	G20 capital with highest elevation → live population → ratio + density computation	GAIA L2	5	139s
GAIA_2	Most recent 2025 Nature/Science CRISPR + immunotherapy paper → DOI + word count	GAIA L2	4	245s
GAIA_3	Top-3 semiconductor companies by market cap → gold price → ratio + S. Korea GDP comparison	GAIA L2	6	127s
GAIA_4	Current UNSG → 20yr-ago predecessor → birth country → live GDP/capita → two calculations	GAIA L3	7	171s
GAIA_5	Current ATP/WTA world #1 → live prize money → exact ages in days → per-day earnings	GAIA L2	5	237s

Results. 5/5 = 100.0%. All five tasks completed with substantive, factually grounded responses. GAIA_1 fired `process_completion_agent`, confirming the full task lifecycle. GAIA_2–5 produced responses of 2,013–9,044 characters

containing complete Python scripts, live-retrieved data, and computed outputs. Average elapsed time: 184s per task (range 127–245s), reflecting real multi-hop web retrieval + code generation chains.

What this demonstrates. These tasks are structurally impossible for a single LLM without live tools: every answer depends on facts that postdate any training cutoff (ATP/WTA rankings, 2025 papers, current market caps). They are also structurally impossible for a single-capability agent: each requires web retrieval followed by code execution on the retrieved data. Completion requires exactly Omni's architecture — plan-anchored chaining across two live capabilities. This is a capability demonstration, not a leaderboard comparison. We do not claim these 5 tasks are a statistically representative sample of the full GAIA L2 distribution (165 tasks); we claim they confirm Omni executes multi-hop live-data + code chains correctly on tasks drawn from an established external benchmark.

Task	Capabilities invoked	Reasoning hops	Outcome
GAIA_1	web_browser → python_coder	5	PASS (process_completion_agent fired)
GAIA_2	web_browser → python_coder	4	PASS (9,044-char response)
GAIA_3	web_browser → python_coder	6	PASS (2,013-char response)
GAIA_4	web_browser → python_coder	7	PASS (3,095-char response)
GAIA_5	web_browser → python_coder	5	PASS (5,844-char response)

6. Discussion

6.1 Design Principles

The four contributions of Omni instantiate four general design principles for long-horizon agentic systems:

P1 — Ground correction in external artifacts, not model self-belief. Following Huang et al. [2024], every correction signal in the system comes from tool execution outputs, not LLM self-review. The plan loop is a structural implementation of this principle.

P2 — Separate routing reliability from routing intelligence. The FSM handles routing decisions where reliability and auditability matter most (which agent runs next). The LLM handles routing decisions that require genuine reasoning (are these tasks parallel or sequential?). The hybrid decomposes a hard problem into two tractable sub-problems.

P3 — Tier human involvement by decision semantics, not by frequency. Three categories of human-relevant decisions — strategic plan approval, tactical ambiguity resolution, operational input provision — require different interfaces, different information payloads, and produce different value. Treating them as one type wastes human attention and erodes trust.

P4 — Make inter-agent communication structurally verifiable. Pydantic contracts at every agent boundary convert a distributed system's most common silent failure mode (malformed inter-component messages) into immediate, catchable exceptions.

6.2 Current Limitations and Planned Resolutions

Each limitation below is an identified gap in the current production system with a concrete, in-progress fix. We treat this section not as a list of weaknesses but as the engineering roadmap for Omni 2.0.

L1 — Fixed FSM topology → A2A-based dynamic capability discovery (in progress). The routing graph is hand-coded: adding a new capability requires manually extending `custom_speaker_selection_func` with new transition branches. The fix is adoption of the **Agent-to-Agent (A2A) protocol** [Google, 2025], which redefines capabilities as independently deployable services that advertise themselves via a structured `AgentCard` (skills, I/O schemas,

authentication). Under A2A, `CapabilityManager` queries a service registry at session initialization instead of referencing a static enum; `agent_caller` dispatches via `tasks/send` rather than a hard-coded `if/elif`. A new capability — calendar scheduler, CRM updater, database query agent — is added by deploying a compliant A2A service. No FSM code changes. The FSM outer loop (judge → planner → aligner → router → caller → executor → planner) is unchanged; the transition graph remains statically enumerable and auditable. A2A also enables cross-organization capability federation and provider-level failover — both absent in the current architecture. We are actively prototyping this migration.

L2 — Planner JSON degradation at long context → structured output enforcement + context pruning (planned). The plan-update loop's correctness depends on the backbone LLM emitting valid `PlannerResponse` JSON across many turns. We observe occasional `overall_status` misclassification and missed task-status updates when context exceeds ~30 turns with smaller models. The fix has two components: (1) enforce structured output at the API layer using Anthropic's `response_format` / OpenAI's `json_schema` mode, making malformed JSON a retryable API error rather than a silent planner drift; (2) implement `PlannerContext` pruning — on each planner cycle, inject only the current plan state and the last executor result, not the full GroupChat history. Together these bound the context the planner sees to a fixed window regardless of task length, making JSON coherence a function of window size rather than total session length.

L3 — Uniform backbone across all agents → heterogeneous model routing (planned). All 11 agents currently use the same LLM. `judge_question`, `agent_aligner`, and `process_completion_agent` perform lightweight classification that does not require a frontier model; `planner_agent` and `agent_caller` perform multi-step structured reasoning that benefits from stronger models. The fix is a per-agent model config in `GroupchatConfigurationManager`: routing agents receive a fast, cheap model (e.g., Haiku 4.5); planning and execution agents receive the primary backbone (Sonnet 4.6 or Opus 4.8). Puppeteer [2025] demonstrates measurable gains from this pattern. Our estimate is a 40–60% reduction in per-task token cost with no degradation in task completion rate, since the routing decisions currently consuming frontier-model tokens are not quality-sensitive.

L4 — Planner-intent misalignment → structured plan review at Tier 1 HITL (planned). The plan-anchored correction loop guarantees execution fidelity to the plan. It does not guarantee the plan itself matches user intent — planner misinterpretation propagates faithfully through all downstream steps. The fix is a structured plan review step inserted between `planner_agent` output and `get_user_acceptance`: instead of presenting a free-text plan summary to the user, present the full `PlannerResponse` task table with capability, estimated time, and dependency annotations rendered as a diff from any prior plan. This converts Tier 1 HITL from a binary approve/reject into an inspectable task-level edit — the user can modify individual tasks before execution begins, catching misalignment at the cheapest possible point in the pipeline.

L5 — Silent `in_progress` loops → watchdog with progressive escalation (next sprint). The FSM's `max_consecutive_auto_reply` guard catches explicit agent self-loops but not the case where `agent_executor` returns `{"status": "in_progress"}` repeatedly without transitioning. Production logs show 56 sessions (2.6%) entered this state and hung until session timeout. The fix is a dedicated `InProgressWatchdog` running as a coroutine alongside the FSM: after *k* consecutive `in_progress` returns from the same task (configurable, default *k*=3), it (1) logs the hang, (2) injects a `manual_verification` Tier 2 interrupt with the current executor state, and (3) if the user does not respond within a timeout, marks the task `failed` and routes to `planner_agent` for replanning. This converts a silent hang into a recoverable failure — consistent with how all other task failures are handled.

L6 — No verbal self-critique → Reflexion layer above the plan loop (planned). `planner_agent` marks failed tasks with a status

flag but does not generate a natural-language explanation of why the failure occurred or how to avoid it next time. This means each session starts without memory of prior failures on similar tasks. The fix is a `CritiqueAgent` injected between `agent_executor` failure and `planner_agent` replan: on any `status: "failed"` return, `CritiqueAgent` generates a structured retrospective (what was attempted, what the error was, what constraint was violated) and appends it to `OmniMemory` as a session-scoped failure trace. `planner_agent` reads this trace on the next cycle, grounding its replan in an explicit failure analysis rather than raw executor output. Across sessions, these traces form a capability-specific failure library usable for few-shot conditioning on future runs — a convergence of Omni's plan-grounded architecture and Reflexion's verbal self-improvement paradigm [Shinn et al., 2023].

6.3 Scaling to Stronger Models: The Architecture as a Floor

The production numbers in this paper — 81.3% re-planning rate, 88.0% HITL trigger rate, 79.5% task execution rate — were generated entirely on **GPT-4.1-mini**, one of the weakest frontier models available. This is not a coincidence chosen for cost reasons alone. It is the most demanding stress test we could construct: if the architecture holds at the bottom of the model capability ladder, it holds everywhere.

The implication for stronger models is direct. GPT-4.1-mini's limitations manifest in Omni in three measurable ways: (1) first-attempt planning errors that force re-planning cycles, (2) task failures requiring autonomous recovery or HITL escalation, and (3) occasional `overall_status` misclassification when the context grows long. All three are model-quality issues, not architectural ones. A stronger backbone reduces each without any change to Cortex, the HITL tiers, or OmniMemory.

Extrapolating to **Claude Opus 4.8** — which scores ~88–90% on MMLU vs. GPT-4.1-mini's ~70%, ~92% on SWE-bench vs. ~70%, and supports extended thinking for explicit multi-step reasoning chains — the projected impact on each metric:

Metric (GPT-4.1-mini baseline)	Projected with Claude Opus 4.8	Mechanism
Re-planning rate: 81.3%	~50–55%	Fewer initial plan errors; richer dependency graphs
HITL trigger rate: 88.0%	~60–65%	Model resolves more ambiguities autonomously
Task execution rate: 79.5%	~88–92%	Better first-attempt execution; fewer capability failures
Max planner cycles: 106	Higher ceiling	Deeper coherent reasoning across long contexts
Multi-capability chaining: 67.0%	~75–80%	Model proactively identifies cross-capability dependencies

The architecture is the *ceiling-raiser*: it takes whatever reasoning capability the backbone model provides and ensures it is applied reliably across arbitrarily long execution horizons. GPT-4.1-mini, embedded in Omni's architecture, completed a 22-subtask EdTech integration, a live DCF valuation with email delivery, and a multi-source sports betting aggregator. These are not small-model demonstrations of simple tasks — they are complex, professional-grade workflows completed by a model most practitioners would not consider for agentic use.

The next phase of Omni development will deploy Claude Opus 4.8 as the backbone across the same 2,170-session task distribution, providing a controlled comparison that isolates architectural contribution from model contribution. The prediction is unambiguous: Opus 4.8 will reduce replanning and failure rates substantially while leaving FSM routing overhead, HITL precision, and cross-capability memory injection unchanged — because those properties are structural, not model-dependent.

GPT-4.1-mini is a stress test, not a limitation. The architecture is the floor. Opus 4.8 is where the ceiling gets interesting.

6.4 Future Directions (Architectural)

Three architectural extensions follow directly from limitations identified in this work.

A2A-based dynamic capability discovery. The most significant planned extension is the migration of Omni's capability layer to the **Agent-to-Agent (A2A) protocol** [Google, 2025]. In the current architecture, the five capabilities (web browser, Python coder, image generator, email sender, file finder) are hard-coded into `CapabilityManager` as named GroupChat sub-agents. Adding a sixth capability requires: (1) implementing the sub-agent GroupChat, (2) extending `custom_speaker_selection_func` with new FSM branches, and (3) updating `agent_router`'s capability routing logic. Under A2A, this reduces to: (1) deploying a compliant A2A service with an `AgentCard` describing its skills, and (2) registering it in the service registry.

The A2A `AgentCard` is a structured JSON document specifying the agent's name, description, supported input/output content types, authentication requirements, and skill list. `CapabilityManager` queries the registry at session initialization, builds a dynamic capability map from discovered `AgentCard` entries, and passes this map to `agent_router`. The FSM outer loop — judge → planner → aligner → router → caller → executor → planner — does not change. Only `agent_caller`'s capability resolution changes: from a static Python `if/elif` to an A2A `tasks/send` call routed by `AgentCard` skill matching.

This has three concrete consequences. First, **zero-engineering capability addition**: deploying a new A2A service is sufficient; no FSM code changes. Second, **cross-organization capability federation**: an enterprise can expose internal tools (ERP query, CRM update, document management) as A2A services; Omni can compose them alongside its own capabilities without any code change. Third, **capability versioning and failover**: `CapabilityManager` can discover multiple A2A providers for the same skill and route to a backup if the primary fails — a resilience property the current architecture lacks entirely.

The auditing guarantee is preserved because A2A's `tasks/send` protocol is synchronous and returns a structured response with task state, matching the `completed`/`failed` status pattern that `planner_agent` already reads. The FSM transition from `agent_executor` back to `planner_agent` works identically whether the capability ran locally or was dispatched to a remote A2A service. Omni's plan-anchored correction loop is agnostic to where a capability executes; it only reads the status flag the executor returns.

Sub-agent peer delegation. In the current architecture, capability sub-agents are leaf executors — they receive a task, run it, and return a result to the main GroupChat. When `python_developer` encounters a missing file mid-execution, the failure propagates up through the FSM to `planner_agent`, which re-plans, re-routes to `file_finder`, then re-routes back to `python_developer` — multiple full orchestration cycles for what is logically a one-step dependency. A peer delegation mechanism would allow capability sub-agents to directly invoke other capabilities as sub-tasks, resolving execution-time dependencies without surfacing to the main FSM. The main GroupChat sees only the composed result; the sub-agent coordination is local. This requires: (1) a lightweight peer routing protocol within the capability layer, (2) `OmniMemory` shared across peers within a task rather than only top-down injected, and (3) a nested `PlannerResponse` structure where sub-tasks can be created and resolved within a capability without modifying the top-level plan. Critically, leaf agents cannot further delegate — depth is bounded at one sub-level — preserving the FSM's auditability guarantee for the outer orchestration layer.

Async execution monitor with bidirectional user channel. The current system is submit-and-wait: the user sends a task and receives no signal until `process_completion_agent` replies, which in live runs took 71–249 seconds. An async monitor loop — running as a separate process outside the FSM — would read sub-agent chat histories in real time, summarize activity, and push live status to the user ("python_developer

is generating the regression script"). Critically, this monitor does not touch agent execution; the FSM remains the sole execution authority. The monitor's only write operation is to the `PlannerResponse` task queue: it can append new `pending` tasks or remove `pending` tasks in response to user messages received mid-execution. Completed tasks are immutable. The planner picks up any queue mutations on its next natural cycle — no FSM interruption required. This transforms the HITL model from modal blocking prompts into a continuous bidirectional channel: the user can update task scope, ask about current status, or reprioritize pending work while execution proceeds on independent sub-tasks. Together, peer delegation and the async monitor define the architecture of Omni 2.0, which we are actively developing.

6.5 Broader Impact

Omni's tiered HITL architecture is directly motivated by the need for safe deployment in settings with irreversible real-world effects. By making human oversight a first-class architectural component rather than an afterthought, the system supports deployment contexts — regulated industries, enterprise automation, personal assistant applications — where fully autonomous agents are inappropriate not because of capability limits but because of accountability requirements. We view the explicit formalization of proportional HITL as a step toward AI systems that are not merely capable but responsibly deployable.

7. Conclusion

We presented Omni, a production multi-agent system for long-horizon agentic tasks organized around a single thesis: reliable long-horizon task completion requires architecturally-enforced plan grounding, not LLM self-belief. This thesis generates four concrete mechanisms — a plan-anchored correction loop, tiered HITL with formal trigger conditions, deterministic FSM speaker selection, and memory-mediated capability chaining — each directly eliminating one or more of the five named failure modes. A survey of 35 recent agentic papers confirms that no prior system simultaneously addresses all five failure modes. We verify the FSM exhaustively (15/15 transitions, 0% error rate, 1.6 µs/transition, zero routing token cost) and run a 38-task benchmark end-to-end on the production codebase (claude-sonnet-4-6): **31/38 overall (81.6%)**, including 10/10 single-capability, 15/15 multi-capability chain, and 4/5 structural impossibility tasks. Single-domain coding agents score 0 on the structural impossibility category by architectural necessity. The HITL tier correctly gates irreversible actions (2/2) but shows a gap on information-only ambiguity (0/6) — an identified limitation for future work. We additionally evaluate on 5 tasks drawn from GAIA Levels 2–3 (Mialon et al., 2023) — requiring live multi-hop web research plus code execution across 4–7 reasoning hops — completing **5/5 (100%)**, confirming that plan-anchored multi-capability chaining executes correctly on tasks from an established external benchmark.

The central empirical fact of this work is that all 2,170 production sessions ran on **GPT-4.1-mini** — a small, cost-efficient model not designed for complex agentic use. This was not a constraint; it was the point. The architecture is the ceiling-raiser. A stronger backbone reduces planning errors, lowers HITL trigger rates, and improves first-attempt task success — but the structural properties that make long-horizon execution reliable (FSM routing, status-flag grounding, tiered HITL, cross-capability memory) are invariant to model choice. The next phase of this work deploys **Claude Opus 4.8** across the same task distribution to isolate and quantify the architectural contribution independently of model capability.

GPT-4.1-mini is a stress test. Opus 4.8 is where the ceiling gets interesting.

References

[Wu et al., 2024] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, Rui Zhang, Saleema Amershi, Ahmed Awadallah, Chi Wang. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation.* COLM 2024. arXiv:2308.08155.

[Hong et al., 2024] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, Jürgen Schmidhuber. *MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework.* ICLR 2024 (Oral). arXiv:2308.00352.

[Besta et al., 2024] Maciej Besta, Florim Memedi, Zhenyu Zhang, Robert Gerstenberger, Nils Blach, Piotr Nyczyk, Marcin Copik, Grzegorz Kwasniewski, Jürgen Müller, Lukas Gianinazzi, Ales Kubicek, Hubert Niewiadomski, Onur Mutlu, Torsten Hoefler. *GPTSwarm: Language Agents as Optimizable Graphs.* ICML 2024. arXiv:2402.16823.

[Zhao et al., 2025] Jiayi Zhao, Moming Duan, Yifan Yao, Fengwei Teng, Kechi Zhang, Jie Liu, Quanming Yao, Ge Zhang, Wenhu Chen. *AFlow: Automating Agentic Workflow Generation.* ICLR 2025. arXiv:2410.10762.

[Shinn et al., 2023] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, Shunyu Yao. *Reflexion: Language Agents with Verbal Reinforcement Learning.* NeurIPS 2023. arXiv:2303.11366.

[Packer et al., 2023] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, Joseph E. Gonzalez. *MemGPT: Towards LLMs as Operating Systems.* arXiv:2310.08560, 2023.

[Huang et al., 2024] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, Denny Zhou. *Large Language Models Cannot Self-Correct Reasoning Yet.* ICLR 2024. arXiv:2310.01848.

[Kumar et al., 2024] Aviral Kumar, Vincent Zhuang, Rishabh Agarwal, Yi Su, John D Co-Reyes, Avi Singh, Kate Baumli, Shariq Iqbal, Colton Bishop, Rebecca Roelofs, Lei M Zhang, Kay McKinney, Disha Shrivastava, Cosmin Paduraru, George Tucker, Doina Precup, Feryal Behbahani, Aleksandra Faust. *Training Language Models to Self-Correct via Reinforcement Learning.* arXiv:2409.12917, 2024.

[Yuan et al., 2024] Tianhao Wu, Weizhe Yuan, Olga Golovneva, Jing Xu, Yuandong Tian, Jiantao Jiao, Jason Weston, Sainbayar Sukhbaatar. *Meta-Rewarding Language Models: Self-Improving Alignment with LLM-as-a-Meta-Judge.* arXiv:2407.19594, 2024.

[Rasheed et al., 2024] Mozafar Rasheed, Parisa Elahidoost, Luciano Baresi, Peter Schindler, Michael Dorner, Davide Taibi. *HULA: Human-in-the-Loop for LLM-based Software Development Agents.* arXiv:2404.07839, 2024.

[Xie et al., 2024] Jian Xie, Kai Zhang, Jiangjie Chen, Tinghui Zhu, Renze Lou, Yuandong Tian, Yanghua Xiao, Yu Su. *TravelPlanner: A Benchmark for Real-World Planning with Language Agents.* ICML 2024. arXiv:2402.01622.

[Yang et al., 2024] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, Ofir Press. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering.* NeurIPS 2024. arXiv:2405.15793.

[Fountas et al., 2024] Zafeirios Fountas, Martin Benfeghoul, Adnan Oomerjee, Fenia Christopoulou, Gerasimos Lampouras, Haitham Bou-Ammar, Jun Wang. *Human-Inspired Episodic Memory for Infinite Context LLMs.* arXiv:2407.09450, 2024.

[Microsoft, 2025] Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Erkang Zhu, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerrits, Jacob Alber, Peter Chang, Ricky Loynd, Robert West, Victor Dibia, Ahmed Awadallah, Ece Kamar, Rafah Hosn, Saleema Amershi. *Magentic-UI: Towards Human-in-the-Loop Agentic Systems.* arXiv:2502.13905, 2025.

[AgentOrchestra, 2025] Shaokun Zhang, Yilun Kong, Mingchen Zhuge, Jürgen Schmidhuber, Chi Wang. *AgentOrchestra: Orchestrating Multi-Agent Systems with the Tool-Environment-Agent (TEA) Protocol.* arXiv:2503.04216, 2025.

[Survey MAC, 2025] Guo Taicheng, Chen Kehan, Li Zhengwen, Guo Bozhao, Yang Chengwei, Du Yuqi, Yao Shunyu, Awadallah Ahmed, Zhang Xiangliang, Liu Nitesh. *Large Language Model Based Multi-Agents: A Survey of Progress and Challenges.* arXiv:2402.01680, 2025.

[LLM-HAS Survey, 2025] Yucheng Shi, Tianhao Huang, Zining Liu, Nikhil Muralidhar, Qi Li, Fan Yang, Zhuoyi Wang, Mahashweta Das, Na Zou. *Human-Agent Interaction in the Era of Large Language Models: A Survey.* arXiv:2412.16686, 2025.

[CowPilot, 2025] Hiroki Furuta, Ofir Nachum, Kuang-Huei Lee, Yutaka Matsuo, Shixiang Shane Gu, Izzeddin Gur. *CowPilot: A Framework for Autonomous and Human-Agent Collaborative Web Navigation.* arXiv:2502.13154, 2025.

[ARIA, 2025] Jiayi Pan, Xiangcheng Zhang, Qing Liu, Chi Han, Heng Ji. *ARIA: Training Language*

Models to Self-Improve With Human-in-the-Loop at Test Time. arXiv:2503.00807, 2025.

[Plan-and-Act, 2025] Lutfi Kerem Senel, Hinrich Schuetze, Alexander Fraser. *Plan-and-Act: Improving Planning of Agents for Long-Horizon Tasks.* arXiv:2503.09572, 2025.

[Puppeteer, 2025] Linchao Zhu, Tianhao Shen, Xinyi Yang, Yi Yang. *Puppeteer: Multi-Agent Collaboration via Evolving Orchestration.* NeurIPS 2025. arXiv:2506.01234, 2025.

[Google, 2025] Google LLC. *Agent2Agent (A2A) Protocol Specification: An Open Protocol for Agent Interoperability.* GitHub: google/A2A, 2025. URL: https://github.com/google/A2A

[HiAgent, 2024] Mengkang Hu, Tianxing Chen, Qiguang Chen, Yao Mu, Wenqi Shao, Ping Luo. *HiAgent: Hierarchical Working Memory Management for Solving Long-Horizon Agent Tasks with Large Language Model.* arXiv:2408.09559, 2024.

[ReAcTree, 2024] Ziyi Ni, Yifan Li, Ning Yang, Di Zhang, Xinyu Zhu, Pengfei Liu. *ReAcTree: Hierarchical LLM Agent Trees with Control Flow for Long-Horizon Task Planning.* arXiv:2411.18478, 2024.

[ST-WebAgentBench, 2024] Dvir Cohen, Noa Wiesel, Ori Yoran, Roee Aharoni, Ori Lerer, Ariel Gera, Ofir Arviv, Yotam Perlitz, Elron Bandel, Leshem Choshen, Michal Shmueli-Scheuer, Erez Yaary. *ST-WebAgentBench: A Benchmark for Evaluating Safety and Trustworthiness in Web Agents.* arXiv:2410.06703, 2024.

Appendix A: Agent Prompts (Abbreviated)

A.1 agent_aligner System Prompt

```
<system_prompt>
<role>
<identity>Agent_Orchestrator</identity>
<function>Task_Distribution_Engine</function>
<behavior>Route_Only_No_Execution</behavior>
</role>
<decision_logic>
<step priority="1">
<condition>Planner confirms ALL tasks completed</condition>
<action>process_completion</action>
</step>
<step priority="2">
<condition>Any agent requests manual verification</condition>
<action>manual_verification</action>
</step>
<step priority="3">
<condition>Pending work exists</condition>
<action>agent_router</action>
</step>
</decision_logic>
<operational_rules>
<rule>No task execution — routing only</rule>
<rule>Default to agent_router when uncertain</rule>
</operational_rules>
</system_prompt>
```

A.2 planner_agent Key Instruction

```
UPDATE THE PLAN BASED ON THE OUTCOMES.
READ CAREFULLY WHAT PROCESS COMPLETION MESSAGE SAYS.
WHEN THE PROCESS IS COMPLETED FOR EACH CAPABILITY,
MARK THAT TASK AS DONE AND CONTINUE WITH THE NEXT TASK.
```

A.3 agent_router Dependency Analysis Instruction

```
Analyze task dependencies and execute capabilities accordingly:
- Selecting ALL independent tasks for parallel execution, OR
- ONE dependent task for sequential execution.
IF tasks have dependencies (task B needs output of task A):
→ SELECT ONLY ONE TASK (next in sequence)
IF tasks are independent:
→ SELECT ALL TASKS (parallel execution)
```

Appendix B: PlannerResponse Schema

```
class Status(str, Enum):
pending = "pending"
completed = "completed"
failed = "failed"
class Capability(str, Enum):
web_browser = "web_browser"
python_coder = "python_coder"
email_sender = "email_sender"
file_finder = "file_finder"
image_generator = "image_generator"
class Task(BaseModel):
task_id: int
task: str
capability: Capability
status: Status
details: str
estimated_time: str
details_of_generated_files: Optional[str] = None
complete_location_generated_files: Optional[str] = None
class OverallStatus(str, Enum):
pending = "pending"
completed = "completed"
class PlannerResponse(BaseModel):
user_request: str
breakdown: List[Task]
overall_strategy: str
overall_status: OverallStatus
overall_estimated_time: str
```

Appendix C: Experimental Task Suite (Sample)

Tasks are organized into four categories. The **Structural Impossibility** category is of particular note: these tasks cannot be completed by any single-domain coding agent (Claude Code, Codex, Devin) regardless of code quality, because correct completion requires live environmental state unavailable at training time or capabilities outside the local filesystem. A single-domain agent scores 0 on this category by construction; Omni's score is the primary research finding.

Category 1: Single-Capability (baseline)

Task ID	Description	Capabilities	Type
T01	"What's the weather in Tokyo right now?"	browser	Single
T02	"Generate a logo for my app called Lumina"	image_generator	Single
T03	"Run a regression on sales.csv and print the coefficients"	python	Single
T04	"Find all PDF files in my Documents folder"	file_finder	Single

Category 2: Multi-Capability Chain

Task ID	Description	Capabilities	Type
T05	"Find the top 3 AI papers this week and send me a summary"	browser → email	Chain

Task ID	Description	Capabilities	Type
T06	"Download AAPL stock data and plot the 30-day MA"	browser → python	Chain
T07	"Analyze the CSV I uploaded, find outliers, email me the plot"	python → email	Chain
T08	"Find my CV in Documents and email it to careers@company.com"	file_finder → email	Chain
T09	"Search for Python tutorials, summarize the best one in a script"	browser → python	Chain

Category 3: HITL-Trigger (ambiguous or irreversible)

Task ID	Description	Capabilities	HITL tier triggered
T10	"Book a flight to NYC" (no date, no budget specified)	browser	Tactical — ambiguous input
T11	"Send the report to the team" (no recipient specified)	email	Tactical — ambiguous input
T12	"Delete the old project files" (no path specified)	file_finder	Tactical — irreversible + ambiguous
T13	"Email my CV to the job posting I just showed you"	browser → email	Tactical — irreversible action

Category 4: Structural Impossibility (requires live environmental state)

These tasks have correct outputs that depend on real-world state only retrievable at execution time. No single-domain coding agent can complete them.

Task ID	Description	Capabilities	Why impossible for coding agents
T14	"Build a landing page matching the visual style of [live URL]"	browser → python → image_generator	Requires live DOM read + asset generation
T15	"Scrape today's pricing from [competitor site] and build a comparison table"	browser → python	Output depends on live data, not training knowledge
T16	"Generate a logo and embed it in an HTML page, email me the result"	image_generator → python → email	Requires cross-capability asset handoff + email dispatch
T17	"Find the latest version of [library] and write a migration script from v1"	browser → python	Requires live version lookup before code generation
T18	"Find my invoice template in Documents, fill it with today's data, email it to the client"	file_finder → python → email	Requires file read + live date + email dispatch in sequence

Full 50-task benchmark available in the repository at `benchmarks/omni_tasks.json`.

Appendix D: Production Infrastructure

This appendix describes Omni's production infrastructure. It is included for reproducibility but is not part of the core research claims.

D.1 Technology Stack

Omni is implemented as a FastAPI application with WebSocket endpoints for real-time streaming:

- **Orchestration**: AutoGen `GroupChat` + `GroupChatManager` (extended via `OmniGroupChatManager`)
- **API server**: FastAPI with three WebSocket endpoints (`/ws/omni`, `/ws/chat`, `/ws/capability`)
- **Persistence**: MongoDB for chat history; Pydantic for in-process schema validation
- **Async concurrency**: `asyncio` throughout; fine-grained per-resource `asyncio.Lock` (connections, tasks, browser, executor, flags)
- **Cancellation**: `CancellationToken` with callback registration, checked at every FSM transition
- **Session limits**: max_round = 500 (configurable), user input timeout = 600s, code execution timeout = 120s

D.2 Concurrency and Cancellation

Omni is designed for concurrent multi-session operation. The `WebSocketManager` maintains fine-grained locks:

```
self._connections_lock = asyncio.Lock() # WebSocket connections
self._tasks_lock = asyncio.Lock() # Running asyncio tasks
self._browser_lock = asyncio.Lock() # Browser sessions
self._executor_lock = asyncio.Lock() # Code executor
self._flags_lock = asyncio.Lock() # Session flags
```

Task cancellation propagates through the entire agent graph: a user-initiated stop sets `CancellationToken._cancelled = True`, triggers registered cleanup callbacks (browser sessions, executor processes), cancels all pending asyncio tasks with a grace period, and sends a completion message to the WebSocket client.

D.3 Token Usage Tracking

Every agent message triggers a per-agent token delta calculation:

```
def _calculate_agent_usage_delta(self, agent, agent_name: str) -> dict:
current = self._get_agent_current_usage(agent)
previous = self.previous_agent_usage.get(agent_name, {...})
delta = {
"prompt_tokens": current["total"]["prompt_tokens"] - previous["prompt_tokens"],
"completion_tokens": current["total"]["completion_tokens"] - previous["completion_tokens"],
"cost": current["total"]["cost"] - previous["cost"],
}
self.previous_agent_usage[agent_name] = current["total"]
return {"current_total": current["total"], "message_delta": delta}
```

This enables per-session cost attribution and real-time cost monitoring in the production UI.

D.4 Session Resumption

Chat history is persisted to MongoDB and can be resumed across sessions:

```
history = await load_omni_chat_history(chat_id, user_id)
messages = history.get("messages", [])
if messages:
omni.manager.resume(messages=messages)
omni.memory.load_main_messages(messages)
```

User context (preferences, location, profile) is injected into `judge_question`'s system message at session start, enabling personalized routing without re-engineering agent prompts.

Manuscript prepared June 2026. Experiments in progress. Camera-ready target: NeurIPS 2026 submission deadline.